A PROJECT REPORT

On

SafeRoute

Submitted by

Prashant Kumar Shiv Kumar Shah

In partial fulfilment for the award of the degree Of

THIRD YEAR

BACHELOR OF SCIENCE

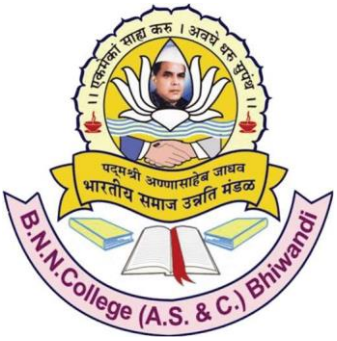
In

COMPUTER SCIENCE

Under the guidance of

Miss. Shilpa Gupta

Department of Computer Science



B.N.N. College, Bhiwandi.

Sem V

A.Y. 2026-2027

Padmashri Annasaheb Jadhav Bhartiya Samaj Unnati Mandal's

B.N.N COLLEGE (ARTS, SCIENCE AND COMMERCE)

(Affiliated to University of Mumbai)

BHIWANDI, MAHARASHTRA-421302

(Self-Funded Courses)

Department of Computer Science

A.Y. 2026-2027

CERTIFICATE

This is to certify that **Mr. Prashant Kumar Shiv Kumar Shah**, of **T.Y. B.Sc. (Sem.-V)** class has satisfactorily completed the Project on "**SafeRoute"** to be submitted in the partial fulfilment for the award of Third Year **Bachelor of Science** in **Computer Science** during the academic year **2026– 2027**.

Date of Submission: 31st October 2026

Internal Guide **Head / In-Charge,**

(Miss. Shilpa Gupta) **Department Computer Science**

College Seal **Signature of Examiner**

DECLARATION BY THE STUDENT

I, **Mr. Prashant Kumar Shiv Kumar Shah** hereby declare that the project entitled "**SafeRoute"** submitted in the partial fulfilment for the award of Third Year **Bachelor of Science** in **Computer Science** during the academic year **2026 – 2027** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student

Place: Bhiwandi

Date: 31th October 2026

ACKNOWLEDGEMENT

I would like to extend our heartiest thanks with a deep sense of gratitude and respect to all those who provides me immense help and guidance during my project work. I would like to thank my project guide of **Miss. Shilpa Gupta** for providing a vision about the system.

I have been greatly benefited from their regular critical reviews and inspiration throughout my work. I am grateful to them for their guidance, encouragement, understanding and insightful support in the development process. I would also like to thank my college for giving required resources whenever I wanted and for giving the opportunity to develop the project.

I would like to express my sincere thanks to our Principal **Dr. Shashikant Mhalunkar** and our Head of Department **Mr. Pramod L. Shewale** for having facilitated us with the essential infrastructure & resources without which this project would not have seen light of the day. I am also thankful to entire staff of Information.

Technology for their constant Encouragement, suggestions and moral support throughout the duration of my project. Last but not the least I would like to mention here that I am greatly indebted to each and everybody my friends and who has been associated with my project at any stage but whose name does not find a place in this acknowledgement.

With Sincere Regards,
Prashant Kumar Shiv Kumar Shah

Safe Routes Stronger Communities

SafeRoute

The shortest path isn't always the safest.

A safety-focused navigation system that helps users find safer routes, report incidents, locate nearby hospitals and police stations, and access emergency assistance.

- Safer Routes
- Report Incidents
- Nearby Hospitals
- Police Stations
- Emergency Assistance



SAFEROUTE

Navigate Smart. Travel Safe.

TABLE OF CONTENTS

Chapter No.	Description	Page No.
1	**INTRODUCTION**	6
1.1	Introduction	
1.2	Objectives	
1.3	Scope of the Project	
1.4	Methodology	
1.5	Hardware Requirements	
1.6	Software Requirements	
2	**FUNCTIONAL REQUIREMENTS**	13
2.1	User Registration and Login	
2.2	Route Search and Navigation	
2.3	Route Safety Score Calculation	
2.4	Incident Reporting	
2.5	SOS Emergency Feature	
2.6	Map and Location Services	
2.7	Safe Havens and Nearby Services	
3	**SYSTEM DIAGRAMS**	17
3.1	System Flowchart	
3.2	Use Case Diagram	
3.3	Activity Diagram	
3.4	Class Diagram	
3.5	Sequence Diagram	
3.6	Deployment Diagram	
3.7	Gantt Chart	
4	**PROGRAM CODE**	24
4.1	Frontend Code	
4.2	Backend Code	
4.3	Database Connectivity Code	
4.4	Route Calculation and Safety Scoring Code	
4.5	Incident Reporting Code	
4.6	SOS Emergency Code	
5	**TESTING**	49
5.1	Testing Approach	
5.2	Functional Testing	
5.3	Test Cases and Results	
5.4	Testing Summary	
6	**ADVANTAGES AND LIMITATIONS**	51
6.1	Advantages	
6.2	Limitations	
7	**PLAGIARISM REPORT**	53
8	**FUTURE SCOPE**	54
9	**CONCLUSION**	55
10	**REFERENCES**	56

INTRODUCTION

1.1 Introduction

SafeRoute is a security-first, community-driven navigation application developed to help users select routes by considering safety along with travel time and distance. Its tagline is “Navigate Smart. Travel Safe.”

Traditional navigation applications generally focus on finding a fast or short route. However, the fastest route may pass through poorly lit streets, isolated areas, or places where safety-related incidents have been reported. Such routes can be a concern for students, pedestrians, tourists, solo commuters, and people travelling at night.

SafeRoute addresses this problem by analysing available route options and safety-related information. It calculates a safety score on a scale of 1 to 10 using the safety metrics implemented in the application. These metrics include relevant incident reports and other supported safety information. The system allows users to compare the fastest and safest available routes and make an informed decision.

The application also provides community-based incident reporting, real-time location sharing through Guardian Mode, an SOS emergency feature, and a map of nearby safe havens such as police stations, hospitals, and pharmacies.

SafeRoute is developed using React, Vite, Tailwind CSS, Node.js, Express.js, MongoDB, and other mapping and communication technologies. It uses map and routing services to display locations and available routes.

The main aim of SafeRoute is to make navigation more safety-aware by helping users understand their route options before travelling. The recommendations depend on available data and cannot guarantee that a route is completely safe.

1.2 Objectives

The main objectives of SafeRoute are:

1. To develop a user-friendly, safety-aware navigation application.
2. To allow users to search for routes between a starting point and a destination.
3. To display and compare different available route options.
4. To calculate safety scores on a scale of 1 to 10 using the implemented scoring method.
5. To identify the safest available route based on the available safety information.
6. To compare the fastest route with the safest route.
7. To consider safety while presenting a faster route when a suitable option is available.
8. To allow users to submit community-based safety incident reports.
9. To provide Guardian Mode for real-time location sharing with selected contacts.
10. To provide an SOS feature for sending emergency information to saved contacts, as supported by the implementation.
11. To display nearby safe havens, including supported police stations, hospitals, and pharmacies.
12. To provide an installable Progressive Web App experience on compatible devices.
13. To protect user accounts through secure authentication methods.
14. To support safer and more informed travel decisions.

1.3 Scope of the Project

SafeRoute is designed for people who want to consider safety when planning their journeys. It combines route planning with available safety information and community-based reporting.

The scope of the project includes the following features.

1. Route Search and Comparison
Users can enter their starting point and destination to retrieve available routes. The system presents route options and allows users to compare their distance, estimated travel time, and safety information.

2. Safety Score Calculation
The application calculates a safety rating from 1 to 10 according to its implemented scoring method. This helps users compare routes based on available safety-related data.

3. Community Incident Reporting
Users can report problems such as poor lighting, theft, harassment, isolated roads, suspicious activity, and other supported incident types. A rate limiter helps restrict repeated reports and reduce spam.

4. Guardian Mode
Guardian Mode supports real-time location streaming while navigation is active. It allows selected contacts to monitor a user's progress when the feature is configured and used appropriately.

5. SOS Emergency Feature
The SOS feature is designed to broadcast the user's last known GPS coordinates and a distress message to saved emergency contacts through the implemented emergency workflow.

6. Safe Havens and Nearby Services
The application can display nearby police stations, hospitals, pharmacies, and other supported locations. Users can explore these locations and request routes to them.

7. Map and Geocoding Services
The application uses mapping, routing, and geocoding services to display maps, find addresses, and obtain route information.

8. Progressive Web App
SafeRoute is designed as an installable PWA, allowing users to access it through compatible mobile and desktop browsers.

9. User Authentication
The system provides registration and login features to protect access to functions that require authentication.

The scope of SafeRoute is limited by the accuracy of location information, availability of external services, and completeness of incident reports. It is a decision-support tool and not a replacement for official emergency services.

1.4 Methodology

The development of SafeRoute followed a systematic software development process.

Step 1: Requirement Analysis
The project's requirements were identified, including route search, route safety scoring, incident reporting, real-time tracking, SOS, and nearby services.

Step 2: System Design
The application architecture, frontend components, backend APIs, database structure, and system diagrams were planned.

Step 3: Frontend Development
React and Vite were used to build the user interface. Tailwind CSS supports styling, while Framer Motion is used for animations and interactive transitions where implemented.

Step 4: Backend Development
Node.js and Express.js were used to develop the backend API for processing application requests, authentication, and other server-side operations.

Step 5: Database Integration
MongoDB and Mongoose were used to store and manage the application data required by the implemented features.

Step 6: Map and Routing Integration
Leaflet and React Leaflet were used for map visualisation. OSRM supports route generation, Nominatim supports address lookup, and the Overpass API supports nearby-place queries where configured.

Step 7: Route Safety Analysis
Available route options are evaluated using the application's implemented safety-scoring method. The system presents route comparisons to help users identify the safest available option and consider travel time.

Step 8: Real-Time and Emergency Features
Socket.IO supports real-time communication for Guardian Mode. The SOS workflow uses the emergency functionality implemented in the application.

Step 9: Security Implementation
JWT is used for token-based authentication, and bcrypt is used for password hashing. Input validation and incident-report rate limiting support safer application operation.

Step 10: Testing and Final Review
The completed application was tested to check its main features, integrations, and expected behaviour. Issues identified during testing were addressed before the project was considered ready for demonstration.

1.5 Hardware Requirements

The following hardware is suitable for developing and running SafeRoute.

Component	Requirement
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB; 8 GB recommended
Storage	Sufficient free space for source code, dependencies, and database files
Display	Standard desktop or laptop display
Input devices	Keyboard and mouse
Network	Internet connection for online routing, maps, and external APIs
Development device	Desktop computer or laptop
Testing device	Compatible smartphone or desktop browser

Actual performance depends on the operating system, browser, available memory, and services used.

1.6 Software Requirements

Software or Technology	Purpose
Windows, Linux, or compatible OS	Operating environment
Visual Studio Code	Source-code editing
HTML, CSS, JavaScript	Web application development
React	Frontend interface
Vite	Frontend development and build tool
Tailwind CSS	User interface styling
Framer Motion	Animations and transitions
Node.js	Backend JavaScript runtime
Express.js	Backend API development
MongoDB	Database
Mongoose	Database modelling and connectivity
Leaflet and React Leaflet	Interactive map display
Socket.IO and Socket.IO Client	Real-time communication
JWT	Token-based authentication
bcrypt	Password hashing
Axios	HTTP requests
OSRM	Route generation
Nominatim	Geocoding and address lookup
Overpass API	Queries for nearby map features
Git and GitHub	Version control and source-code hosting
Modern web browser	Running and testing the application
HTTPS tunnel tool, such as ngrok	Optional secure access for local mobile testing

The project also requires correctly configured environment variables and access to the external services used by the application.

FUNCTIONAL REQUIREMENTS

Functional requirements describe the features provided by SafeRoute and explain how users interact with the application.

2.1 User Registration and Login

SafeRoute provides user registration and login functionality to manage user access.

User Registration

1. The user opens the registration page.
2. The user enters the required information.
3. The frontend validates the input.
4. The registration request is sent to the backend.
5. The backend processes the request and stores the account information in MongoDB.
6. The application displays the appropriate response.

User Login

1. The user enters their login credentials.
2. The frontend sends the login request to the backend.
3. The backend verifies the credentials.
4. If the credentials are valid, the user receives an authentication token.
5. The application allows access to the relevant features.

JWT supports token-based authentication, while bcrypt supports secure password hashing. These mechanisms help protect accounts when correctly configured.

2.2 Route Search and Navigation

Route search is one of the most important features of SafeRoute. It helps users find and compare routes according to travel time, distance, and available safety information.

The process works as follows:

1. The user enters the starting point and destination.
2. The application uses its geocoding and routing services to obtain location and route information.
3. Available route options are retrieved.
4. The application analyses relevant safety-related data for the available routes.
5. The implemented scoring method calculates the safety ratings.
6. The system identifies the safest available route.
7. It compares the fastest and safest options.
8. A faster route that also considers safety can be presented when a suitable option is available.
9. The routes are displayed on the interactive map.
10. The user compares the options and selects a preferred route.

The system does not assume that the shortest route is always the safest. It helps users understand the differences between route options before making a decision.

2.3 Route Safety Score Calculation

SafeRoute uses a safety-scoring engine to evaluate available route options. Each route receives a safety rating on a scale of 1 to 10, according to the implemented method.

The score helps users compare the relative safety of different routes.

The scoring process includes:

1. Retrieving available routes.
2. Collecting relevant safety-related information.
3. Analysing available incident reports and other supported metrics.
4. Applying the implemented weighting and scoring method.
5. Calculating a safety rating for each route.
6. Comparing the ratings of the available routes.
7. Identifying the route with the strongest safety rating.
8. Presenting alternative routes so the user can consider travel time and safety together.

For example, consider two routes to the same destination. Route A may take less time but pass through an area with more reported incidents. Route B may take slightly longer but have a better safety rating. SafeRoute allows the user to compare these options.

The system can distinguish between:

- Fastest route: The route with the lowest estimated travel time among the available options.
- Safest available route: The route with the strongest safety rating under the implemented scoring method.
- Faster, safety-conscious alternative: A route that offers a useful balance between travel time and safety when such an option is available.

A safety rating is an estimate based on available information. It cannot guarantee that a route is completely safe. The quality of the rating depends on the scoring method and the accuracy and freshness of the underlying data.

2.4 Incident Reporting

The incident reporting feature allows users to contribute information about safety-related problems in their surroundings. Supported incident categories include:

- Poor lighting
- Theft
- Harassment
- Isolated road or area
- Suspicious activity
- Road hazard

The reporting process is as follows:

1. The user opens the incident reporting section.
2. The user selects the incident category.
3. The user enters the required description and location details.
4. The application validates the submitted information.
5. The frontend sends the report to the backend.
6. The backend processes the report.
7. The report is stored in the database if it satisfies the application's validation requirements.
8. The user receives an appropriate response.

SafeRoute includes a 10-minute rate-limiting rule for incident reporting to help reduce repeated submissions and spam. Incident reports can support route safety analysis when they are relevant to the routes being evaluated. Reports should not automatically be considered verified facts unless a verification process is provided.

2.5 SOS Emergency Feature

The SOS emergency feature is designed to provide a quick way for users to initiate an emergency-support action. According to the project's documented functionality, the SOS workflow broadcasts the user's last known GPS coordinates and a distress message to saved emergency contacts.

The general process is:

1. The user opens SafeRoute.
2. The user activates the SOS feature.
3. The application obtains the last known GPS coordinates available to the feature.
4. The system prepares the distress message and location information.
5. The SOS workflow sends the information to the saved emergency contacts through the configured communication mechanism.
6. The application presents the relevant response or status.

The actual delivery of an SOS message depends on the implementation, network connectivity, and availability of the configured communication service.

Users should not rely exclusively on the application during an emergency. They should contact local emergency services directly when immediate assistance is required.

2.6 Map and Location Services

SafeRoute uses interactive maps to display geographical information and route options.

The main functions include:

- Displaying the map interface.
- Showing the origin and destination.
- Drawing available route paths.
- Displaying location markers.
- Supporting route comparison.
- Obtaining location information when supported and permitted.
- Displaying nearby places and safe havens.
- Supporting real-time location features when enabled.

Leaflet and React Leaflet are used for map visualisation. OpenStreetMap provides map data, while the configured routing and geocoding services support route generation and location searches.

GPS accuracy can vary according to the device, surroundings, and available signals. Location-dependent features may also require user permission.

2.7 Safe Havens and Nearby Services

SafeRoute includes a feature for locating useful nearby services and designated safe places.

Examples include:

- Police stations
- Hospitals
- Pharmacies
- Other supported nearby services

The Overpass API can be used to query relevant OpenStreetMap features, while the application's configured data sources determine which places are available.

The process works as follows:

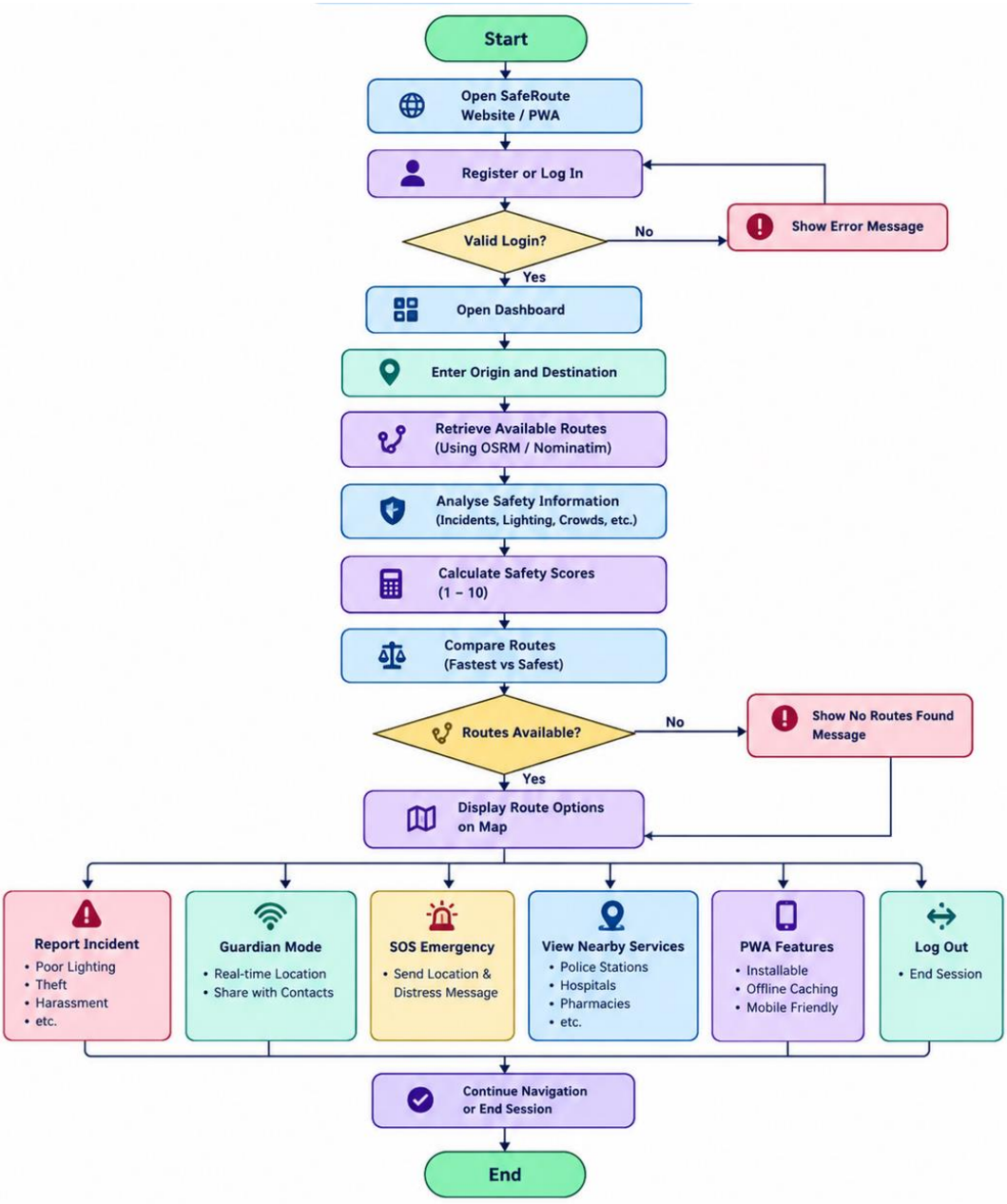
1. The user opens the safe havens or nearby services section.
2. The application obtains or uses the relevant location.
3. A query is made to the configured source.
4. Available nearby locations are retrieved.
5. The locations are displayed on the map.
6. The user can select a location and request a route to it.

This feature helps users find useful places when planning a journey or when they need to locate a nearby service. The information may be incomplete or outdated, so users should verify the availability and details of important services when necessary.

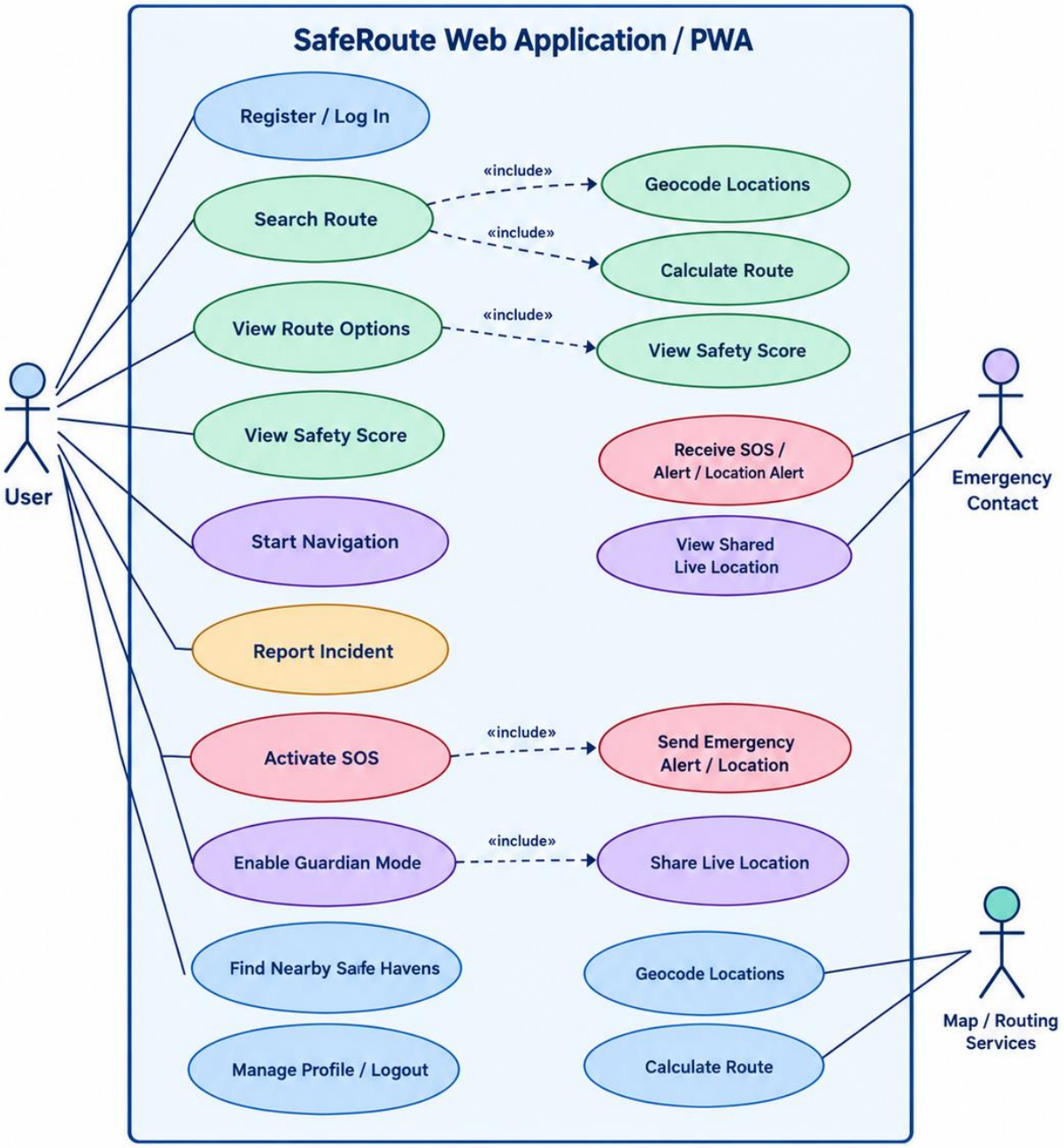
SYSTEM DIAGRAMS

System diagrams explain the workflow, structure, and interactions of SafeRoute. They help readers understand how the application operates without examining all the source code.

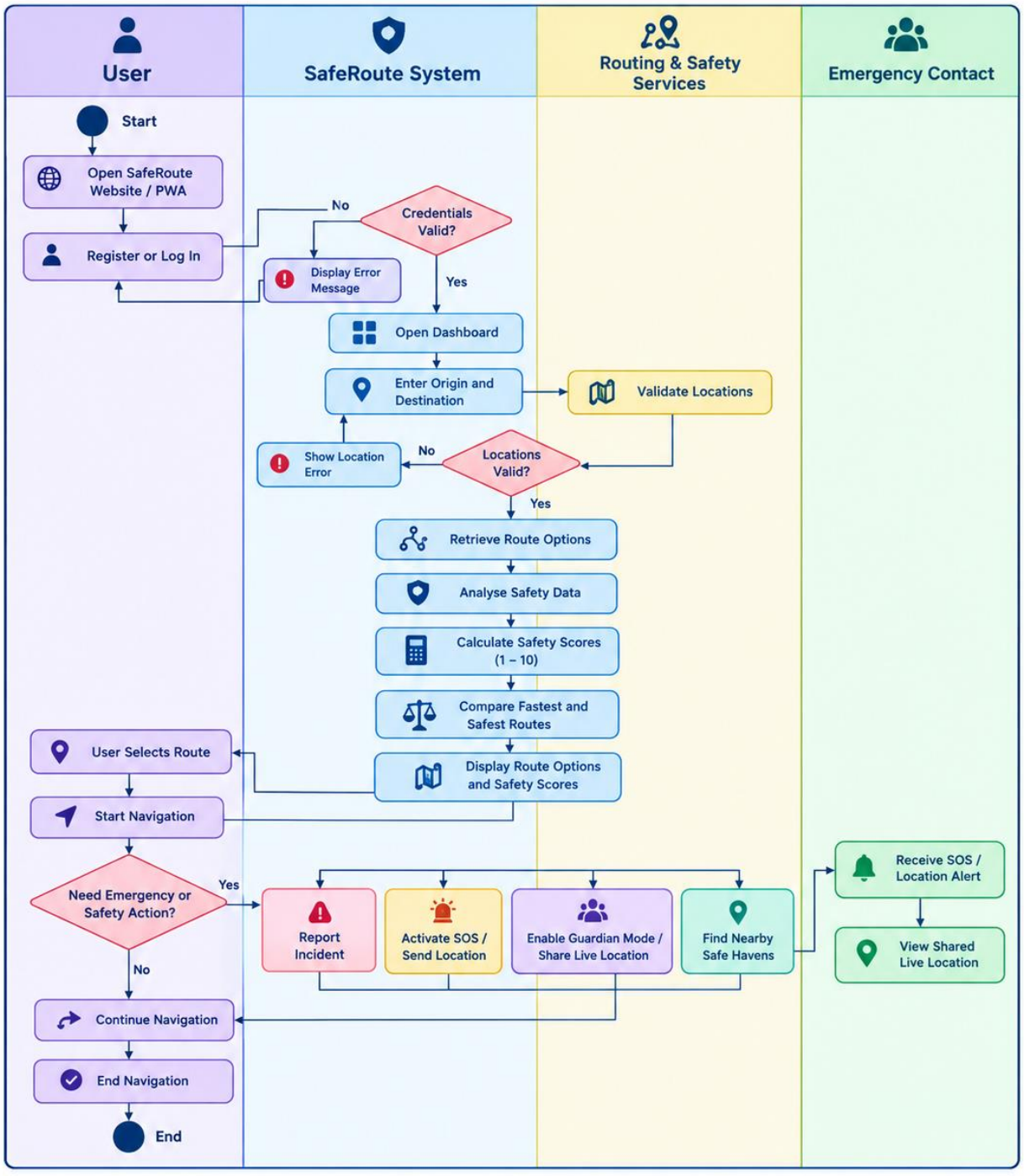
3.1 System Flowchart



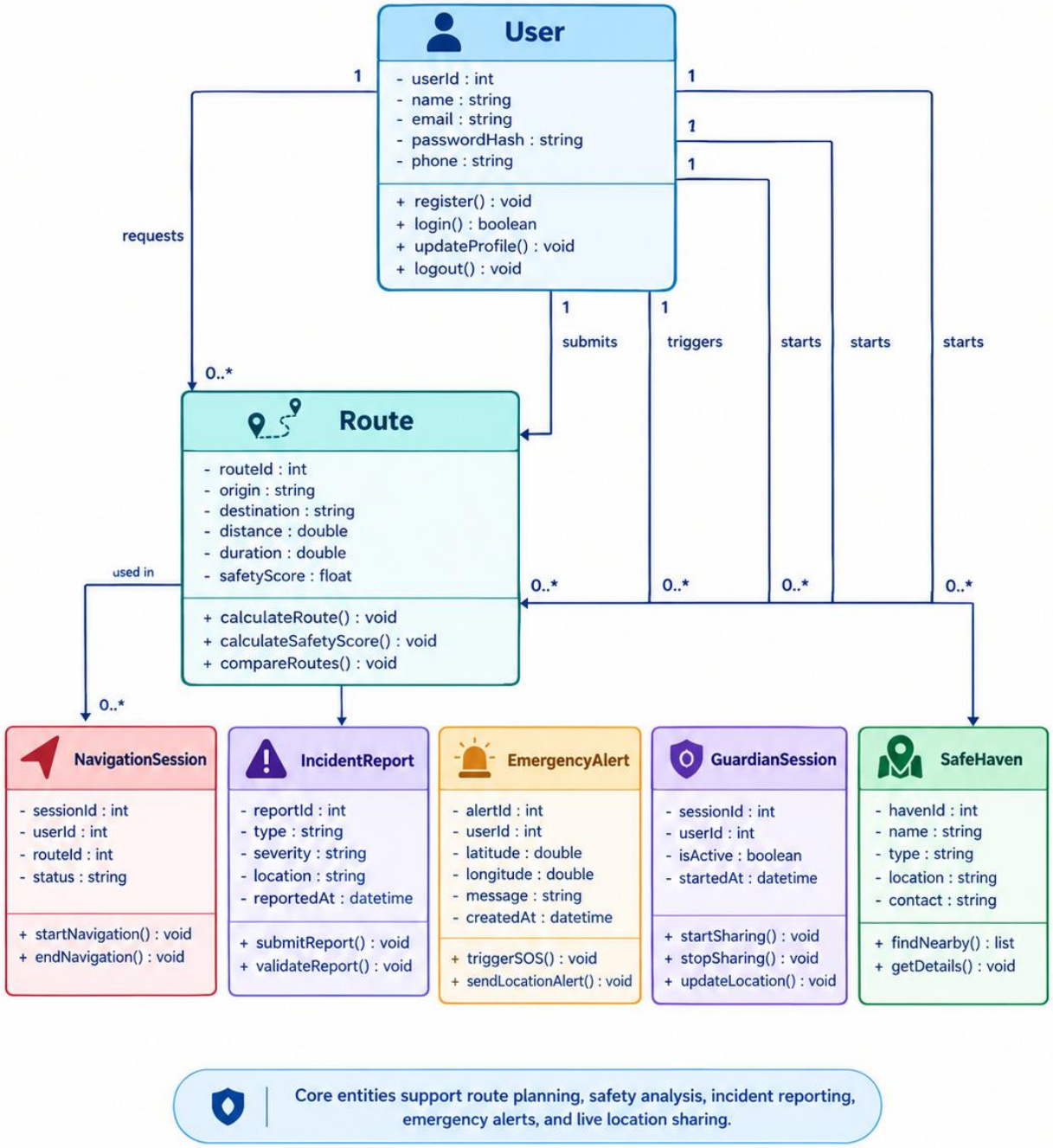
3.2 Use Case Diagram



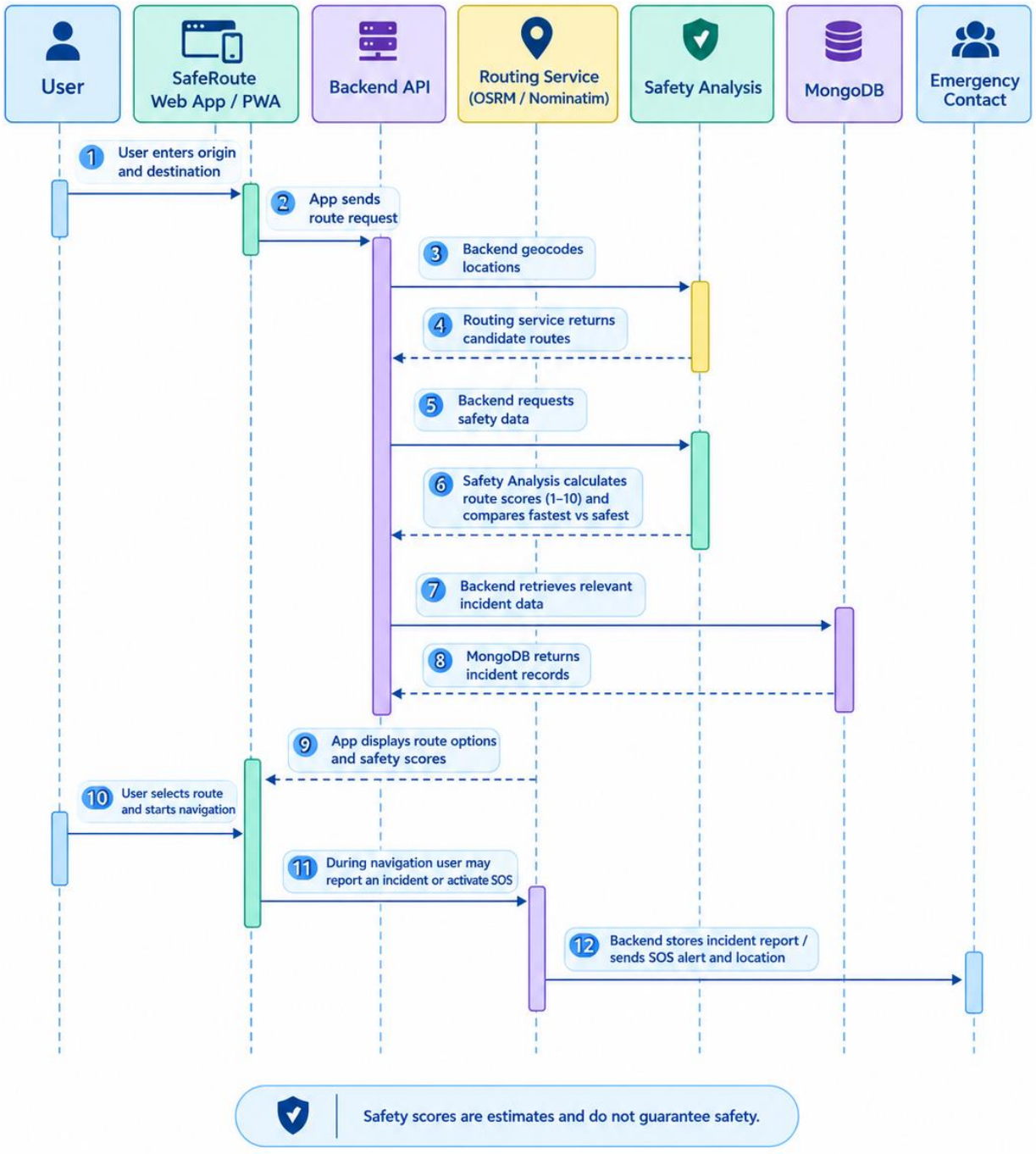
3.3 Activity Diagram



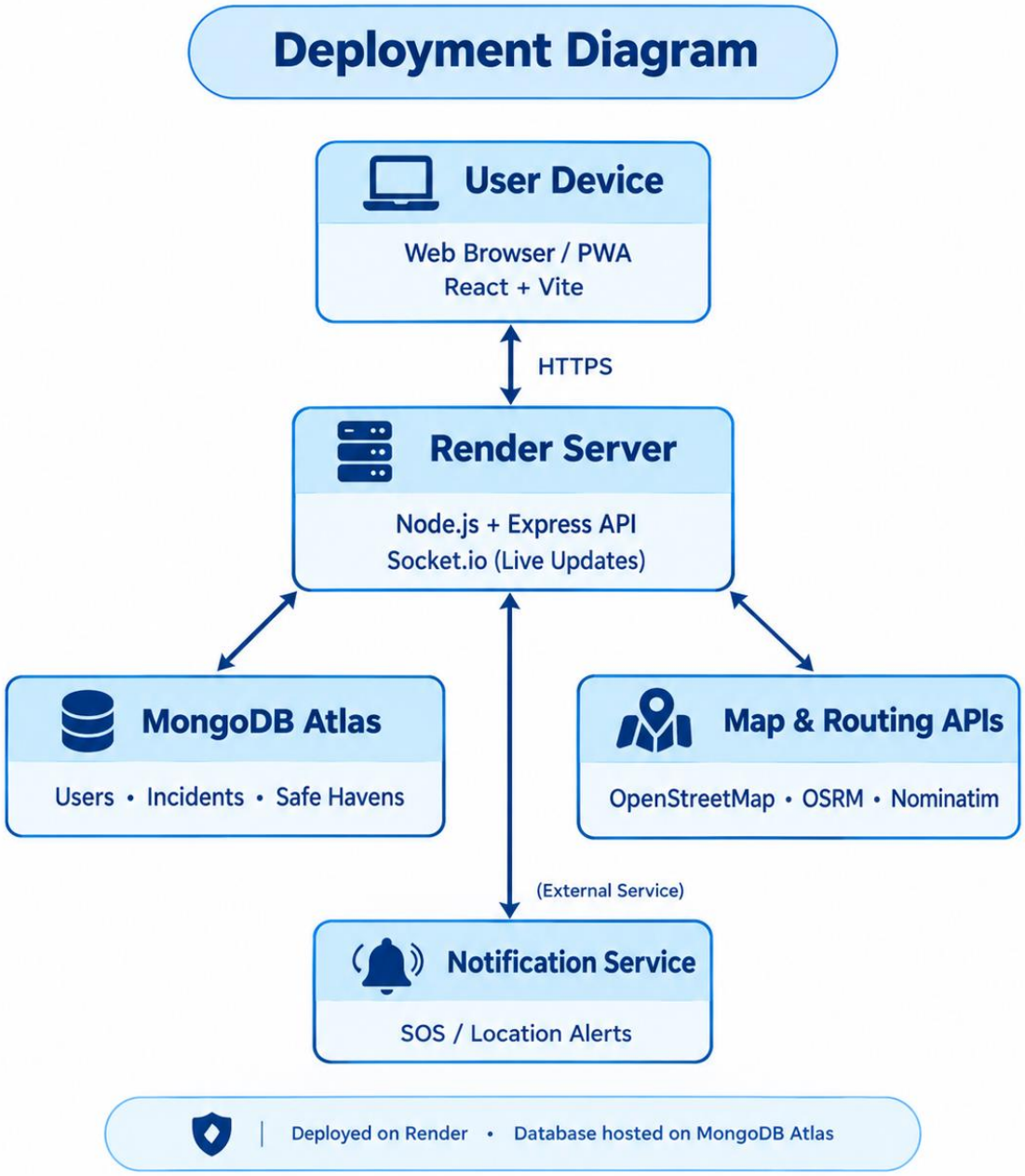
3.4 Class Diagram



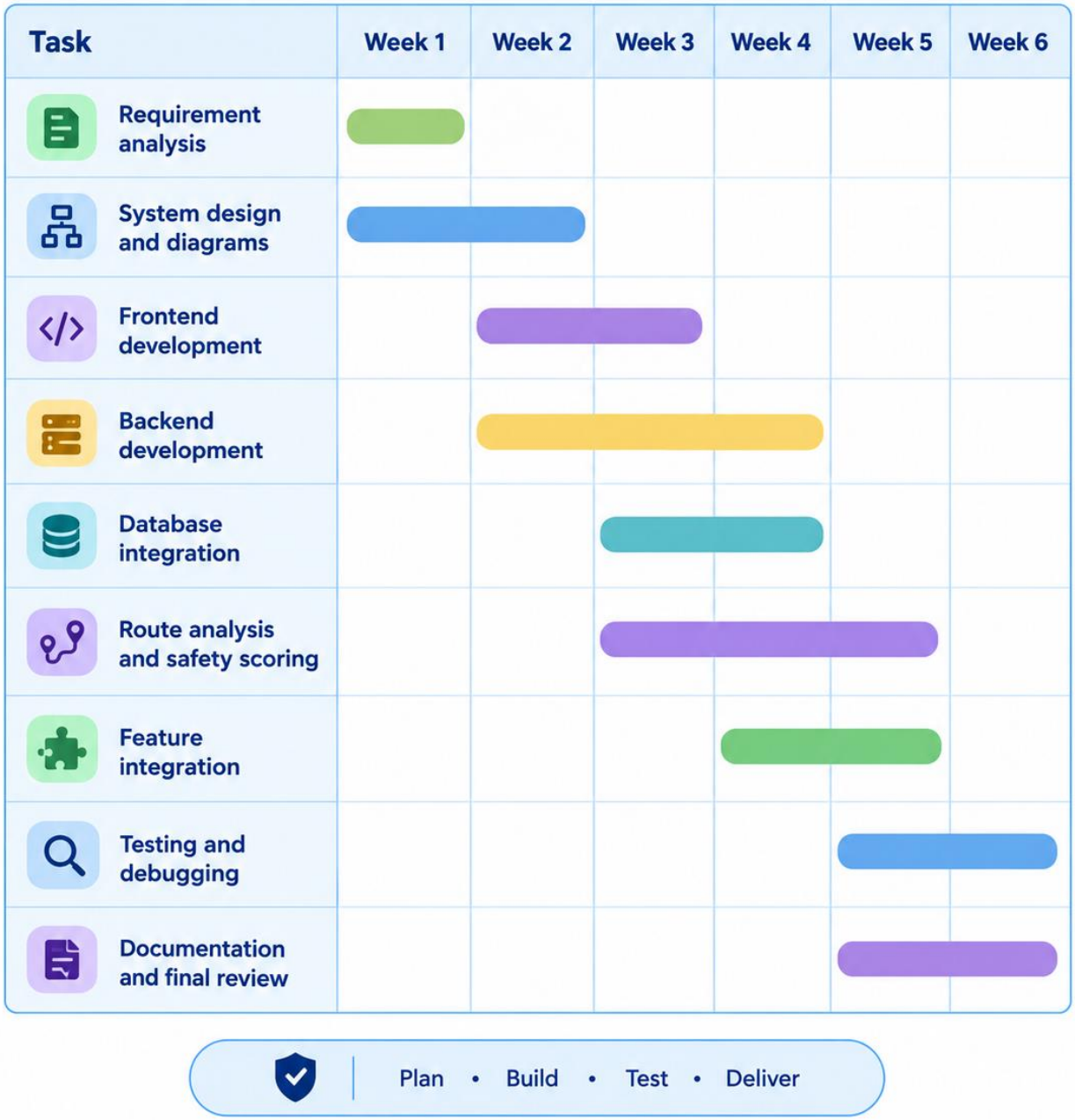
3.5 Sequence Diagram



3.6 Deployment Diagram



3.7 Gantt Chart



PROGRAM IMPLEMENTATION

4.1 Frontend Implementation

1) Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>client</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

2) App.jsx

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import Landing from './pages/Landing';
import Auth from './pages/Auth';
import AppLayout from './pages/AppLayout';
import RouteSearch from './pages/RouteSearch';
import Incidents from './pages/Incidents';
import Contacts from './pages/Contacts';
import 'leaflet/dist/leaflet.css';

function ProtectedRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();
  if (loading) {
    return (
      <div style={{
        display: 'flex', alignItems: 'center', justifyContent: 'center',
        height: '100vh', background: 'var(--background)', color: 'var(--primary)',
        fontSize: 20, gap: 12
      }}>
        🛡️ Loading SafeRoute...
      </div>
    );
  }
  return isAuthenticated ? children : <Navigate to="/auth" replace />;
}
```

```
function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          {/* Public */}
          <Route path="/" element={<Landing />} />
          <Route path="/auth" element={<Auth />} />

          {/* Protected App */}
          <Route
            path="/app"
            element={
              <ProtectedRoute>
                <AppLayout />
              </ProtectedRoute>
            }
          >
            <Route index element={<RouteSearch />} />
            <Route path="incidents" element={<Incidents />} />
            <Route path="contacts" element={<Contacts />} />
          </Route>

          {/* Catch-all */}
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;
```

3) App.css

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
```

```
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;
```

```
  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
```

```
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }

  @media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;

    li {
      flex: 1 1 calc(50% - 8px);
    }

    a {
      width: 100%;
      justify-content: center;
      box-sizing: border-box;
    }
  }
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
```

```
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

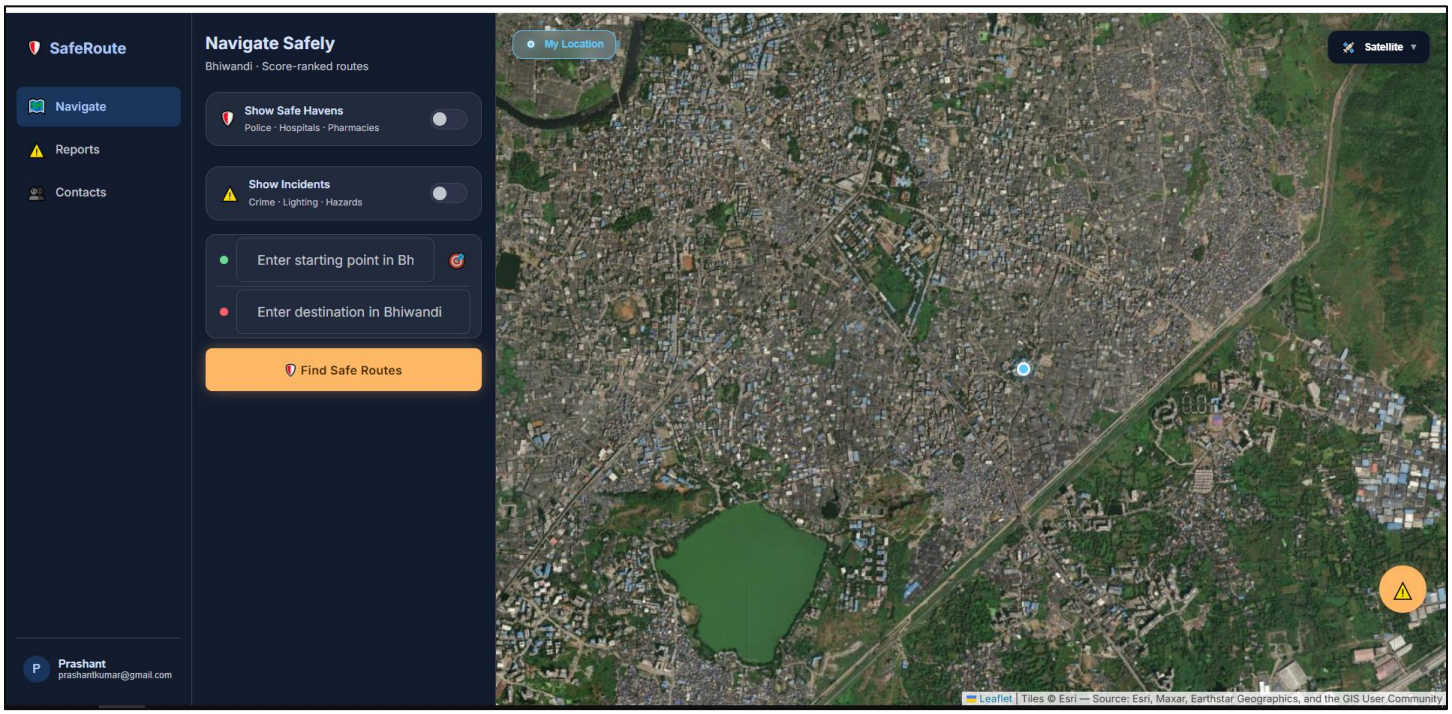
  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}
```

4) main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output:



4.2 Backend Code

Index.js

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
const connectDB = require('./config/db');

connectDB();

const app = express();
const server = http.createServer(app);

const rawClientUrl = process.env.CLIENT_URL || 'http://localhost:5173';
const CLIENT_URL = rawClientUrl.endsWith('/') ? rawClientUrl.slice(0, -1) : rawClientUrl;

const io = new Server(server, {
  cors: {
    origin: CLIENT_URL,
    methods: ['GET', 'POST'],
  },
});

// In-memory last-known positions: { userId: { lat, lng, timestamp } }
const userPositions = new Map();

io.on('connection', (socket) => {
  console.log(`🔌 Socket connected: ${socket.id}`);

  // Client streams their live position
  socket.on('position:update', (data) => {
    const { userId, lat, lng } = data;
    if (userId && lat && lng) {
      userPositions.set(userId, { lat, lng, timestamp: new Date(), socketId: socket.id });
    }
  });

  // SOS uses last known position if real-time coord not available
  socket.on('sos:request_position', (userId) => {
    const pos = userPositions.get(userId);
    socket.emit('sos:position_response', pos || null);
  });

  socket.on('disconnect', () => {
    console.log(`🔌 Socket disconnected: ${socket.id}`);
  });
});
```

```
app.set('userPositions', userPositions);

app.use(cors({
  origin: CLIENT_URL,
  credentials: true,
}));
app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: false }));

if (process.env.NODE_ENV === 'development') {
  app.use((req, _res, next) => {
    console.log(`${new Date().toISOString().substr(11, 8)} ${req.method} ${req.path}`);
    next();
  });
}

app.use('/api/auth', require('./routes/auth'));
app.use('/api/routes', require('./routes/routes'));
app.use('/api/incidents', require('./routes/incidents'));
app.use('/api/contacts', require('./routes/contacts'));
app.use('/api/sos', require('./routes/sos'));
app.use('/api/feedback', require('./routes/feedback'));
app.use('/api/safe-havens', require('./routes/safeHavens'));

app.get('/health', (_req, res) => {
  res.json({
    status: 'ok',
    service: 'SafeRoute API',
    timestamp: new Date().toISOString(),
    mongodb: require('mongoose').connection.readyState === 1 ? 'connected' : 'disconnected',
  });
});

app.use((_req, res) => {
  res.status(404).json({ success: false, message: 'Route not found.' });
});

app.use((err, _req, res, _next) => {
  console.error('Unhandled error:', err);
  res.status(500).json({ success: false, message: 'Internal server error.' });
});

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`\n🛡️  SafeRoute API running on port ${PORT}`);
  console.log(`📡 Socket.io live tracking enabled`);
  console.log(`🗺️  OSRM: ${process.env.OSRM_BASE_URL || 'http://router.project-osrm.org'}`);
  console.log(`🌍 Client: ${CLIENT_URL}\n`);
});

module.exports = { app, server };
```

4.3 Database Connectivity Code

db.js

```
const mongoose = require('mongoose');

const MONGODB_URI = process.env.MONGODB_URI;

if (!MONGODB_URI) {
  console.error('❌ MONGODB_URI environment variable is not set.');
  process.exit(1);
}

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(MONGODB_URI, {
      serverSelectionTimeoutMS: 15000,
      socketTimeoutMS: 45000,
    });

    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error('❌ MongoDB connection error:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Output:

```
PS D:\SafeRoute\server> npm run dev

> server@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injected env (10) from .env // tip: ⌁ auth for agents [www.vestauth.com]

🚨 SafeRoute API running on port 5000
📡 Socket.io live tracking enabled
🗺️ OSRM: http://router.project-osrm.org
🌎 Client: http://localhost:5173

✅ MongoDB Connected: ac-3h1di1n-shard-00-02.ebfkapu.mongodb.net
```

4.4 Route Calculation and Safety Scoring Code

safetyScoring.js

```
/**
 * safetyScoring.js — SafeRoute Safety Scoring Engine
 * Safety score:
 * 40% Community Incident Reports
 * 30% Street Lighting / Road Type
 * 20% Time of Day
 * 10% Road Classification
 * Final score: 0.0 - 10.0
 */

const Incident = require("../models/Incident");

const INCIDENT_RADIUS_METERS = 250;
const INCIDENT_LIFETIME_DAYS = 30;
const MAX_ROUTE_SAMPLES = 80;

const getTimeOfDayScore = (hour) => {
  if (hour >= 6 && hour < 10) return 0.90;
  if (hour >= 10 && hour < 16) return 1.00;
  if (hour >= 16 && hour < 20) return 0.85;
  if (hour >= 20 && hour < 22) return 0.70;
  if (hour >= 22 && hour < 24) return 0.55;
  if (hour >= 0 && hour < 2) return 0.45;
  if (hour >= 2 && hour < 4) return 0.35;
  if (hour >= 4 && hour < 6) return 0.50;
  return 0.80;
};

const getRoadTypeScore = (legs) => {
  if (!Array.isArray(legs) || legs.length === 0) return 0.55;

  const roadTypeWeights = {
    motorway: 0.95,
    trunk: 0.90,
    primary: 0.85,
    secondary: 0.80,
    tertiary: 0.70,
    residential: 0.60,
    service: 0.45,
    footway: 0.30,
    path: 0.25,
    unclassified: 0.50,
    default: 0.55,
  };
```

```
 let totalDuration = 0;
  let weightedScore = 0;

  legs.forEach((leg) => {
    if (!Array.isArray(leg.steps)) return;

    leg.steps.forEach((step) => {
      const name = String(step.name || "").toLowerCase();
      let score = roadTypeWeights.default;

      if (
        name.includes("sector") ||
        name.includes("main road") ||
        name.includes("highway") ||
        name.includes("nh ") ||
        name.includes("nh-") ||
        name.includes("marg") ||
        name.includes("station road")
      ) {
        score = roadTypeWeights.primary;
      } else if (
        name.includes("market") ||
        name.includes("chowk") ||
        name.includes("nagar") ||
        name.includes("bazaar") ||
        name.includes("road")
      ) {
        score = roadTypeWeights.secondary;
      } else if (
        name === "" ||
        name.includes("lane") ||
        name.includes("gali") ||
        name.includes("path") ||
        name.includes("footway")
      ) {
        score = roadTypeWeights.footway;
      }

      const duration = Number(step.duration) || 1;
      weightedScore += score * duration;
      totalDuration += duration;
    });
  });

  return totalDuration <= 0 ? 0.55 : weightedScore / totalDuration;
};

const getLightingScore = (roadTypeScore, hour) => {
  const isNight = hour < 6 || hour >= 20;
  if (!isNight) return roadTypeScore;
```

```
const nightPenalty = 0.70;
  return roadTypeScore * nightPenalty + (1 - nightPenalty) * 0.20;
};

const getSeverityMultiplier = (severity) => {
  switch (severity) {
    case "high": return 1.50;
    case "medium": return 1.00;
    case "low": return 0.55;
    default: return 1.00;
  }
};

const getIncidentTypeMultiplier = (type) => {
  switch (type) {
    case "harassment": return 1.50;
    case "theft": return 1.40;
    case "suspicious_activity": return 1.30;
    case "isolated_area": return 1.20;
    case "poor_lighting": return 1.15;
    case "road_hazard": return 1.00;
    default: return 1.00;
  }
};

const getRecencyFactor = (incident) => {
  const date = incident.reportedAt || incident.createdAt;
  if (!date) return 0.5;

  const ageMs = Date.now() - new Date(date).getTime();
  const ageDays = ageMs / (1000 * 60 * 60 * 24);

  if (!Number.isFinite(ageDays)) return 0.5;
  if (ageDays <= 0) return 1.0;
  if (ageDays >= INCIDENT_LIFETIME_DAYS) return 0;

  return Math.max(0, 1 - ageDays / INCIDENT_LIFETIME_DAYS);
};

const getIncidentConfidence = (incident) => {
  const confidence =
    incident.confidenceScore ??
    incident.confidence ??
    0.5;

  const value = Number(confidence);
  if (!Number.isFinite(value)) return 0.5;

  return Math.max(0, Math.min(1, value));
};
```

```
const sampleRouteCoordinates = (routeCoords) => {
  if (!Array.isArray(routeCoords)) return [];

  const validCoords = routeCoords.filter(
    (coord) =>
      Array.isArray(coord) &&
      coord.length >= 2 &&
      Number.isFinite(Number(coord[0])) &&
      Number.isFinite(Number(coord[1]))
  );

  if (validCoords.length <= MAX_ROUTE_SAMPLES) return validCoords;

  const samples = [];
  const step = (validCoords.length - 1) / (MAX_ROUTE_SAMPLES - 1);

  for (let i = 0; i < MAX_ROUTE_SAMPLES; i++) {
    samples.push(validCoords[Math.round(i * step)]);
  }

  return samples;
};

const getIncidentScore = async (routeCoords) => {
  if (!Array.isArray(routeCoords) || routeCoords.length === 0) return 1.0;

  const sampledCoords = sampleRouteCoordinates(routeCoords);
  if (sampledCoords.length === 0) return 1.0;

  const uniqueIncidents = new Map();

  for (const coord of sampledCoords) {
    const lng = Number(coord[0]);
    const lat = Number(coord[1]);

    try {
      const nearbyIncidents = await Incident.find({
        location: {
          $near: {
            $geometry: {
              type: "Point",
              coordinates: [lng, lat],
            },
            $maxDistance: INCIDENT_RADIUS_METERS,
          },
        },
        isActive: true,
      }).select(
        "_id type severity confidence confidenceScore reportedAt createdAt confirmations"
      );

      for (const incident of nearbyIncidents) {
```

```
        const id = incident._id.toString();
        if (!uniqueIncidents.has(id)) {
          uniqueIncidents.set(id, incident);
        }
      }
    } catch (queryError) {
      console.error("Incident query failed:", queryError.message);
    }
  }

  if (uniqueIncidents.size === 0) return 1.0;
  let totalPenalty = 0;

  for (const incident of uniqueIncidents.values()) {
    const confidence = getIncidentConfidence(incident);
    const recency = getRecencyFactor(incident);
    const severity = getSeverityMultiplier(incident.severity);
    const typeMultiplier = getIncidentTypeMultiplier(incident.type);

    const confirmations = Number(incident.confirmations) || 0;
    const confirmationFactor = Math.min(
      1.20,
      1 + confirmations * 0.05
    );

    const incidentPenalty =
      confidence *
      recency *
      severity *
      typeMultiplier *
      confirmationFactor;

    totalPenalty += incidentPenalty;
  }

  const routeLengthFactor = Math.max(1, sampledCoords.length / 10);
  const normalizedPenalty = totalPenalty / routeLengthFactor;

  const score = 1 - Math.min(0.90, normalizedPenalty);

  return Math.max(0.10, Math.min(1.0, score));
};

const scoreRoute = async (routeCoords, legs, hour) => {
  const safeHour = Number.isFinite(Number(hour))
    ? Number(hour)
    : new Date().getHours();

  const [incidentScore, roadTypeScore] = await Promise.all([
    getIncidentScore(routeCoords),
    Promise.resolve(getRoadTypeScore(legs)),
  ]);
```

```
  const timeScore = getTimeOfDayScore(safeHour);
  const lightingScore = getLightingScore(roadTypeScore, safeHour);

  const composite =
    0.40 * incidentScore +
    0.30 * lightingScore +
    0.20 * timeScore +
    0.10 * roadTypeScore;

  const finalScore = composite * 10;

  return Math.round(finalScore * 10) / 10;
};

const getScoreLabel = (score) => {
  if (score >= 8.5) {
    return { label: "Very Safe", color: "green" };
  }

  if (score >= 7.0) {
    return { label: "Safe", color: "green" };
  }

  if (score >= 5.5) {
    return { label: "Moderate", color: "yellow" };
  }

  if (score >= 4.0) {
    return { label: "Use Caution", color: "yellow" };
  }

  return { label: "High Risk", color: "red" };
};

const getScoreExplanation = async (routeCoords, legs, hour) => {
  const incidentScore = await getIncidentScore(routeCoords);
  const roadTypeScore = getRoadTypeScore(legs);
  const timeScore = getTimeOfDayScore(hour);
  const lightingScore = getLightingScore(roadTypeScore, hour);
  const isNight = hour < 6 || hour >= 20;

  return {
    incident: {
      score: Math.round(incidentScore * 10) / 10,
      weight: "40%",
      label: "Community Incident Reports",
    },
```

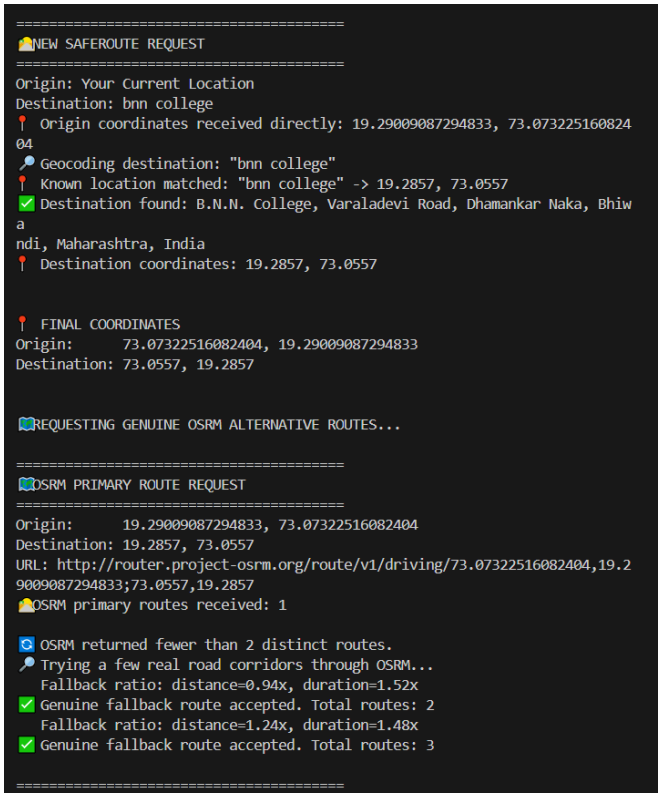
```
  lighting: {
      score: Math.round(lightingScore * 10) / 10,
      weight: "30%",
      label: `Street Lighting${
        isNight ? " (night penalty applied)" : ""
      }`,
    },

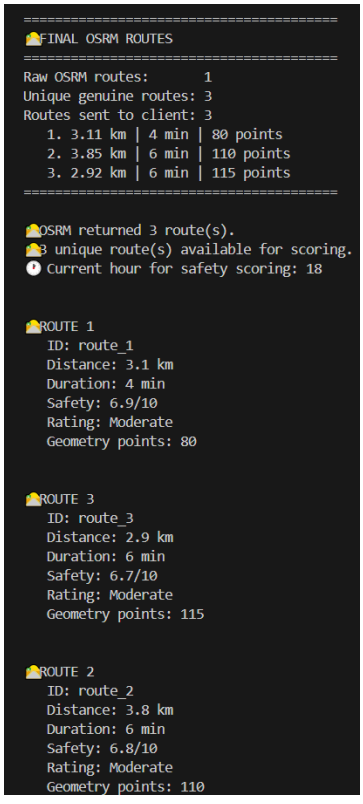
    timeOfDay: {
      score: Math.round(timeScore * 10) / 10,
      weight: "20%",
      label: `Time of Day (${hour}:00)`,
    },

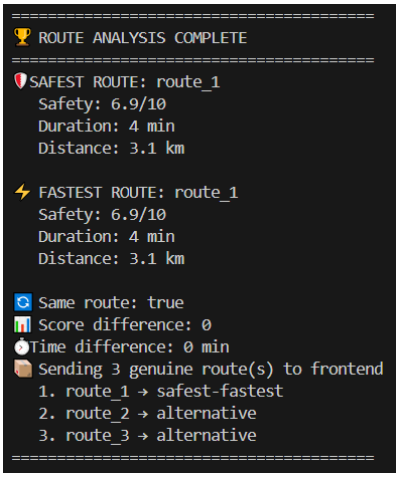
    roadType: {
      score: Math.round(roadTypeScore * 10) / 10,
      weight: "10%",
      label: "Road Classification",
    },
  };
};

module.exports = {
  scoreRoute,
  getScoreLabel,
  getScoreExplanation,
  getTimeOfDayScore,
};
```

Output:







4.5 Incident Reporting Code

Incident.js

```
const mongoose = require('mongoose');

const INCIDENT_TYPES = ['poor_lighting', 'theft', 'harassment', 'isolated_area', 'suspicious_activity', 'road_hazard'];

const incidentSchema = new mongoose.Schema({
  title: { type: String, required: [true, 'Incident title is required'], trim: true, maxlength: 100 },
  type: { type: String, enum: INCIDENT_TYPES, required: [true, 'Incident type is required'] },
  severity: { type: String, enum: ['low', 'medium', 'high'], default: 'medium', required: [true, 'Incident severity is required'] },
  description: { type: String, maxlength: [500, 'Description too long'], trim: true },
  location: {
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: {
      type: [Number], // [longitude, latitude]
      required: [true, 'Location coordinates are required'],
      validate: {
        validator: (coords) => coords.length === 2 && coords[0] >= -180 && coords[0] <= 180 && coords[1] >= -90 && coords[1] <= 90,
        message: 'Invalid coordinates',
      },
    },
  },
  reportedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  source: { type: String, default: 'SafeRoute Community Report' },
  confidenceScore: { type: Number, default: 0.5, min: 0, max: 1 },
  // Sync with confidenceScore for scoring engine backward compatibility
  confidence: { type: Number, default: 0.5, min: 0, max: 1 },
  affectsGroups: { type: [String], default: [] },
  reportedAt: { type: Date, default: Date.now },
  confirmations: { type: Number, default: 0 },
  confirmedBy: { type: [mongoose.Schema.Types.ObjectId], ref: 'User', default: [] },
  // Auto-expiry: weight decays after 30 days
  expiresAt: {
    type: Date,
    default: () => new Date(Date.now() + 30 * 24 * 60 * 60 * 1000), // 30 days
    index: { expireAfterSeconds: 0 }, // MongoDB TTL index
  },
  isActive: { type: Boolean, default: true },
}, { timestamps: true });

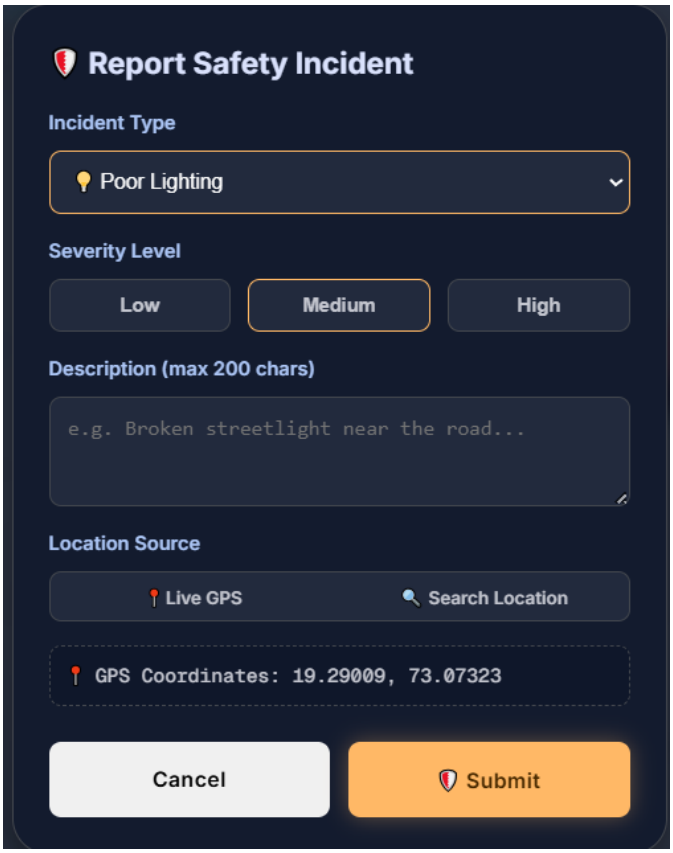
// Pre-save hook to ensure legacy confidence is always in sync with confidenceScore
incidentSchema.pre('save', function () {
  if (this.isModified('confidenceScore')) this.confidence = this.confidenceScore;
  else if (this.isModified('confidence')) this.confidenceScore = this.confidence;
  else if (this.confidenceScore !== undefined) this.confidence = this.confidenceScore;
```

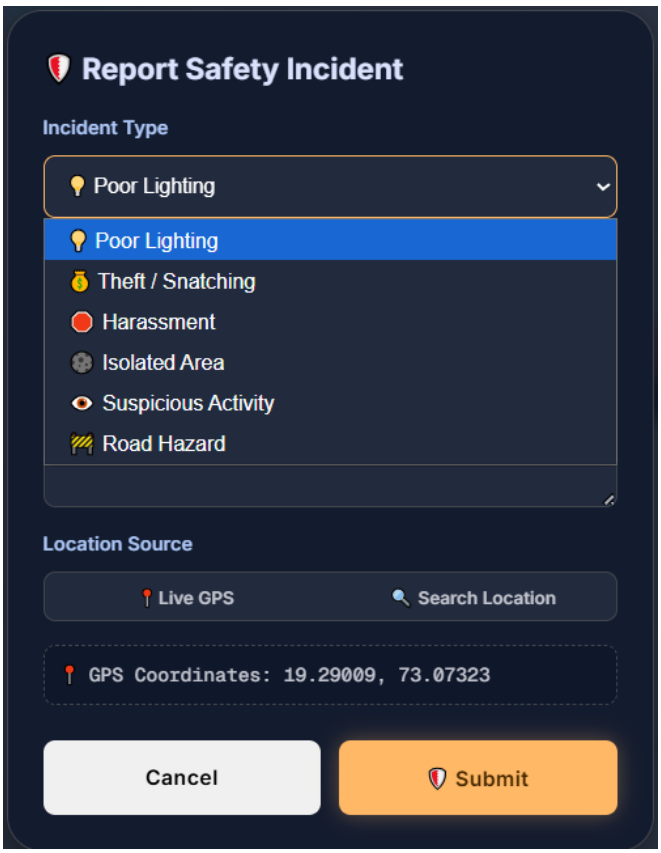
```
});

// Geospatial index for proximity queries
incidentSchema.index({ location: '2dsphere' });
incidentSchema.index({ createdAt: -1 });

module.exports = mongoose.model('Incident', incidentSchema);
module.exports.INCIDENT_TYPES = INCIDENT_TYPES;
```

Output:





4.6 SOS Emergency Code

SosAlert.js

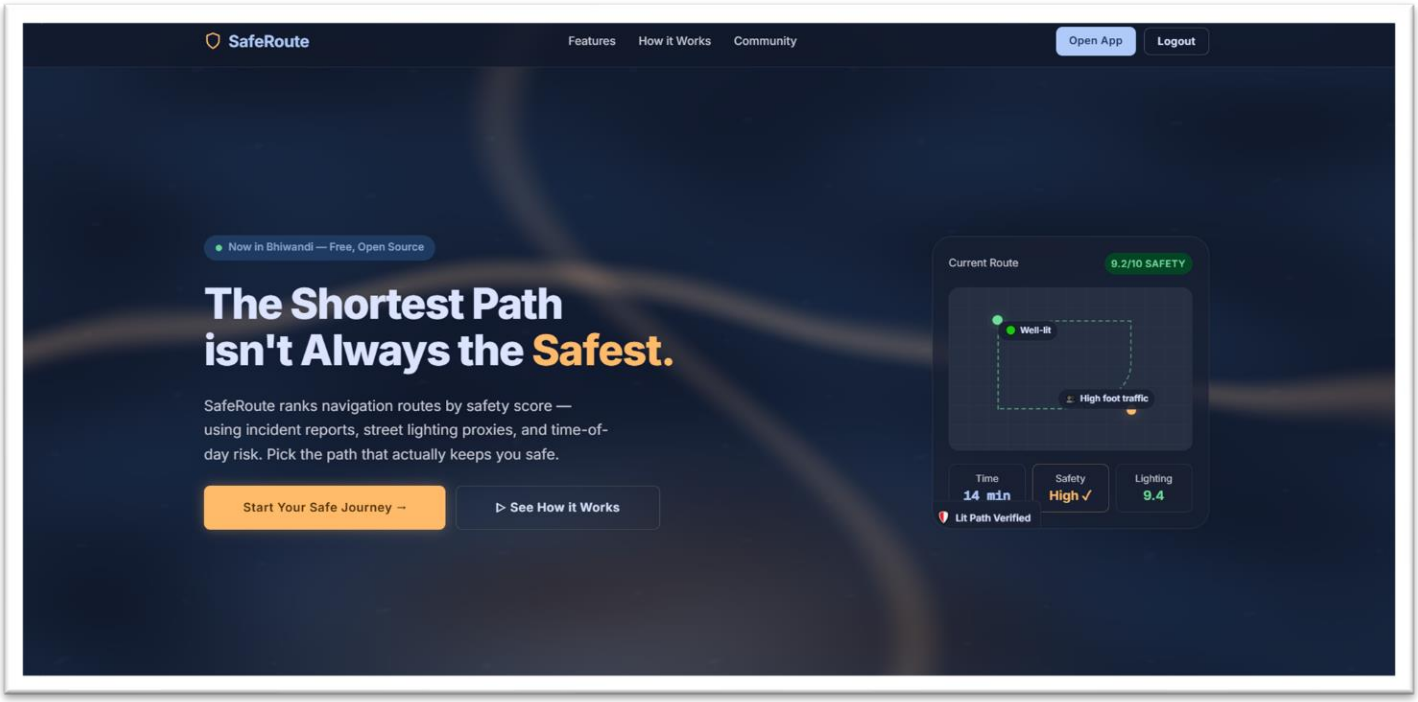
```
const mongoose = require('mongoose');

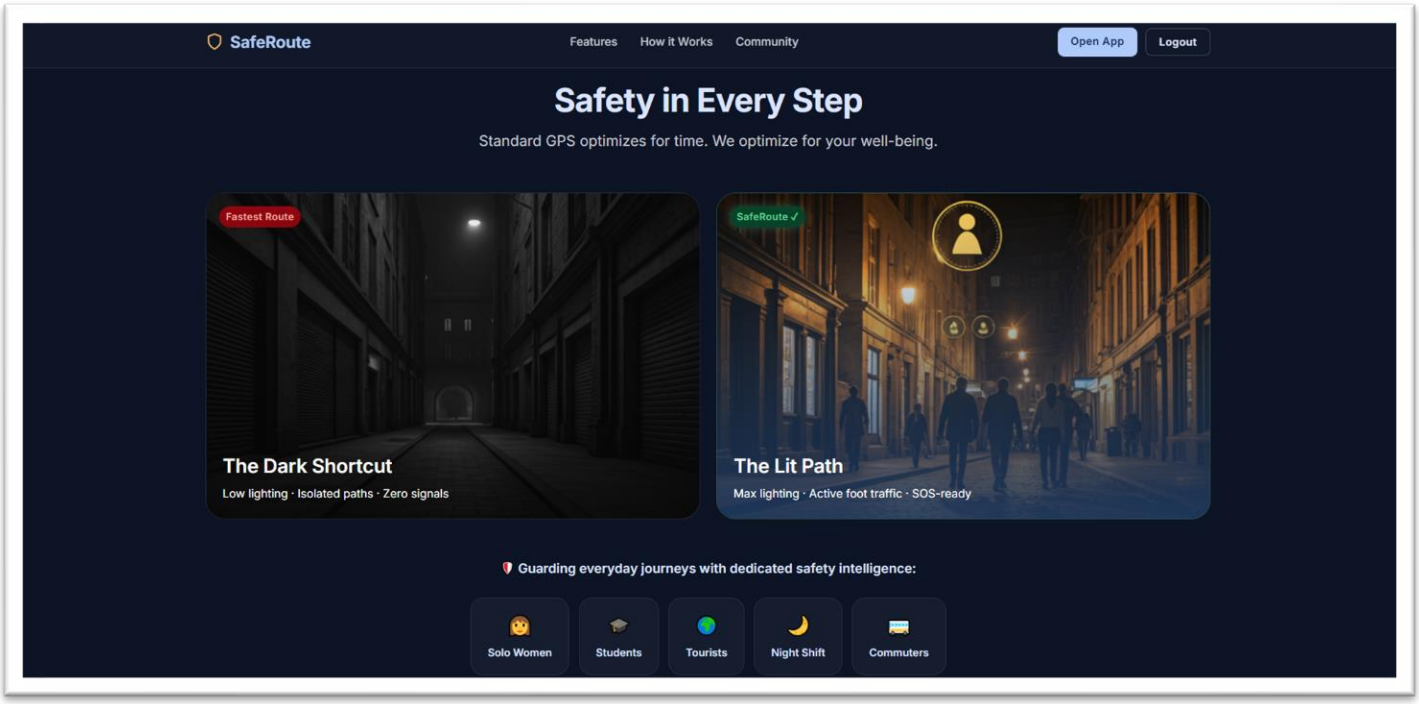
const sosAlertSchema = new mongoose.Schema({
  triggeredBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
  userName: String,
  userPhone: String,
  location: {
    lat: { type: Number, required: true },
    lng: { type: Number, required: true },
  },
  osmLink: String, // openstreetmap.org link with coordinates
  contactsNotified: [
    {
      name: String,
      phone: String,
      relationship: String,
    },
  ],
  message: {
    type: String,
    required: true,
  },
  status: {
    type: String,
    enum: ['sent', 'acknowledged', 'resolved'],
    default: 'sent',
  },
}, { timestamps: true });

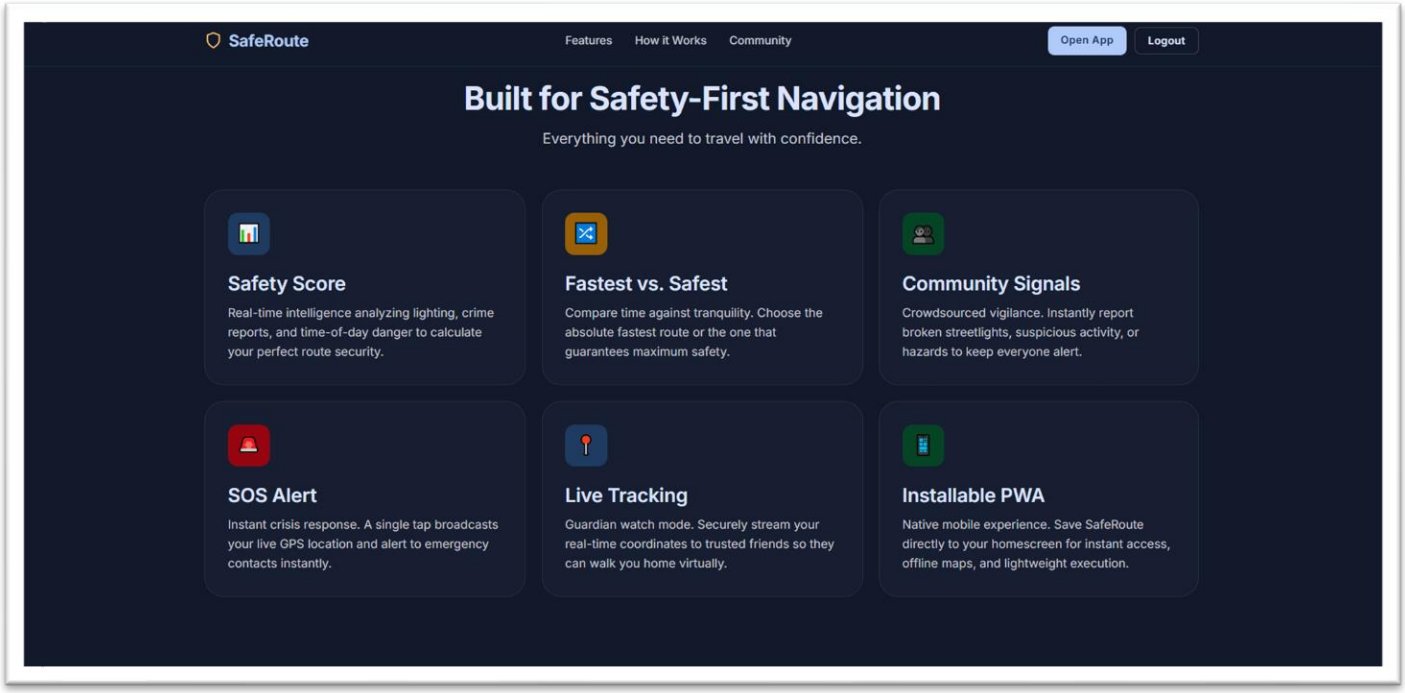
module.exports = mongoose.model('SosAlert', sosAlertSchema);
```

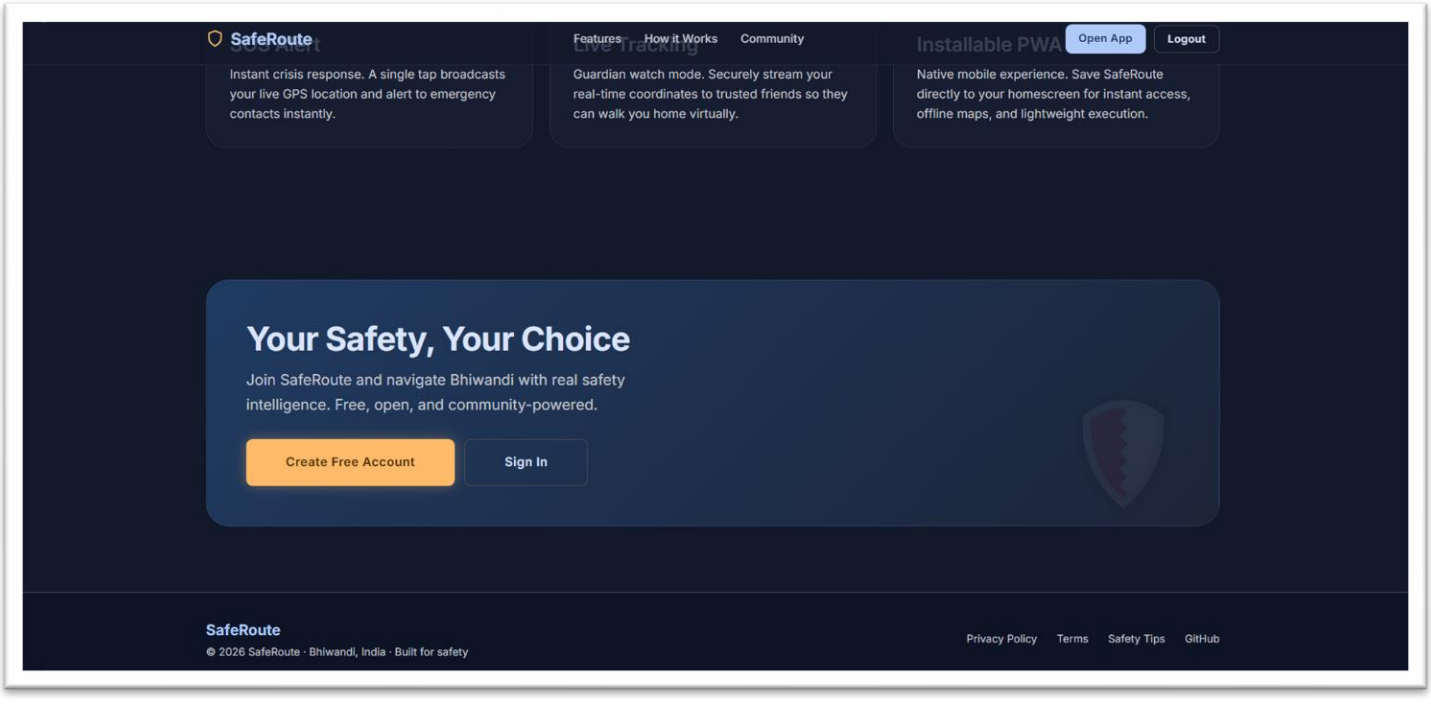
Overall Output:

1) Landing Page

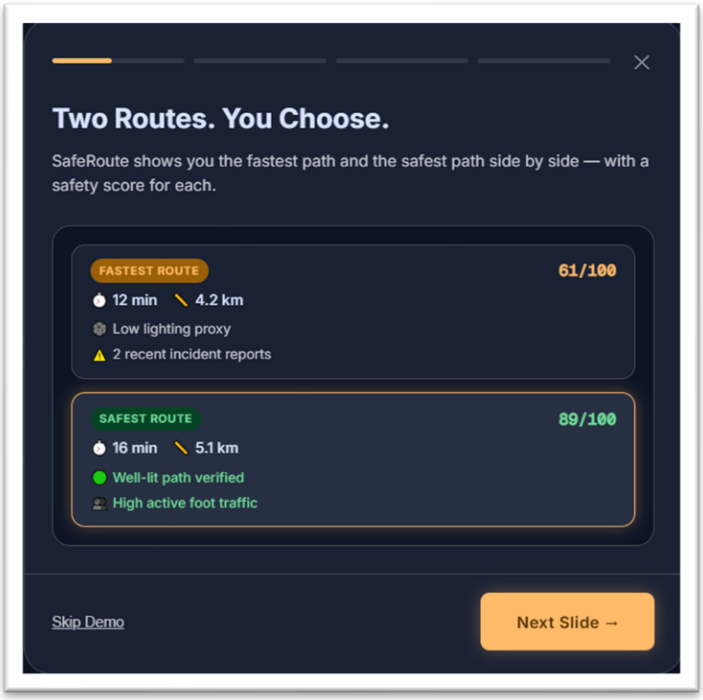


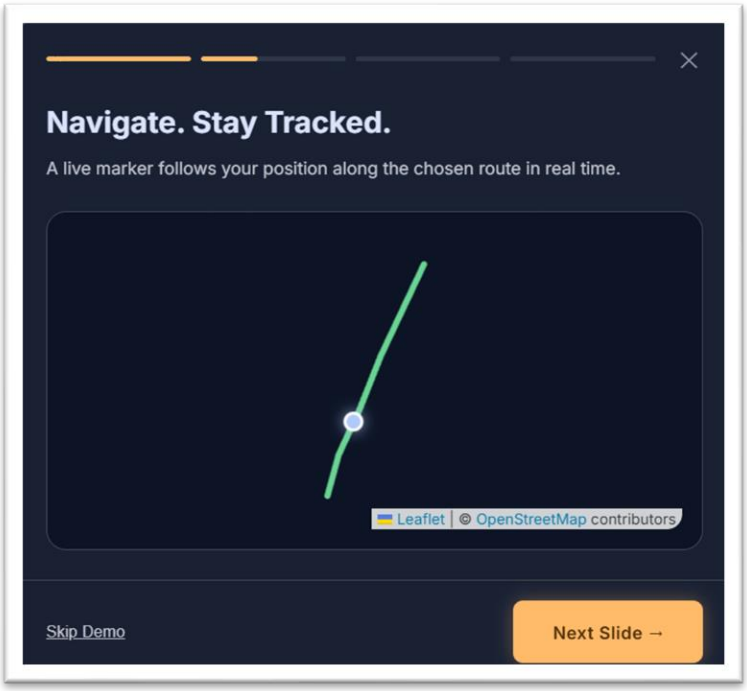


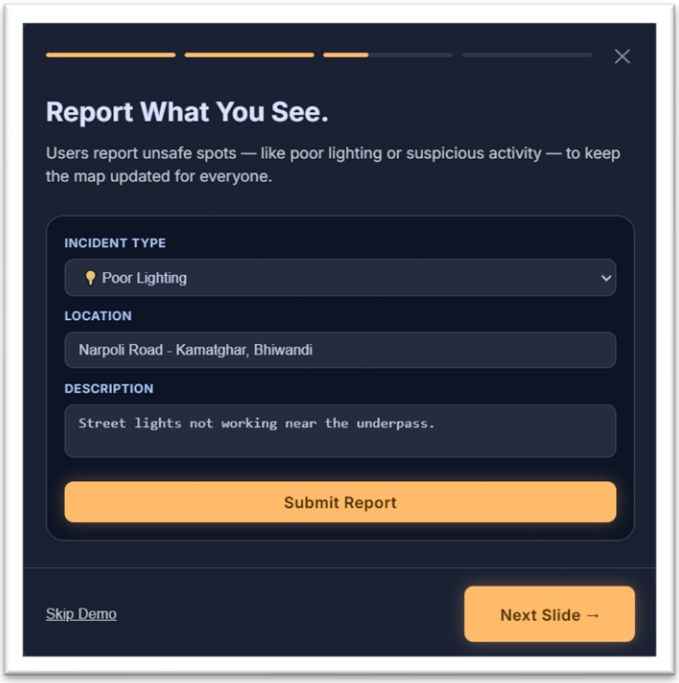

SafeRoute
Features
How it Works
Community
Open App
Logout
Built for Safety-First Navigation
Everything you need to travel with confidence.
Safety Score
Real-time intelligence analyzing lighting, crime reports, and time-of-day danger to calculate your perfect route security.
Fastest vs. Safest
Compare time against tranquility. Choose the absolute fastest route or the one that guarantees maximum safety.
Community Signals
Crowdsourced vigilance. Instantly report broken streetlights, suspicious activity, or hazards to keep everyone alert.
SOS Alert
Instant crisis response. A single tap broadcasts your live GPS location and alert to emergency contacts instantly.
Live Tracking
Guardian watch mode. Securely stream your real-time coordinates to trusted friends so they can walk you home virtually.
Installable PWA
Native mobile experience. Save SafeRoute directly to your homescreen for instant access, offline maps, and lightweight execution.

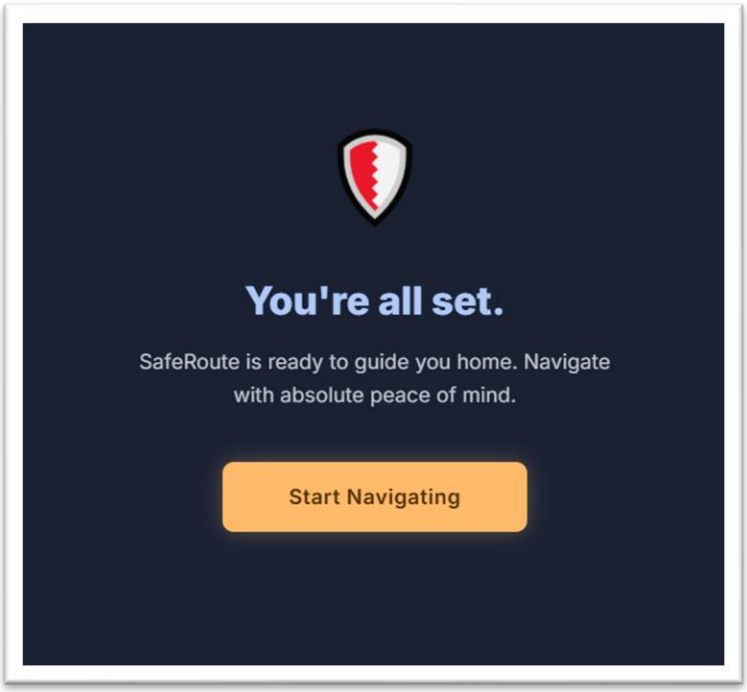

SafeRoute
Features
How it Works
Community
Open App
Logout
Instant crisis response. A single tap broadcasts your live GPS location and alert to emergency contacts instantly.
Guardian watch mode. Securely stream your real-time coordinates to trusted friends so they can walk you home virtually.
Native mobile experience. Save SafeRoute directly to your homescreen for instant access, offline maps, and lightweight execution.
Your Safety, Your Choice
Join SafeRoute and navigate Bhiwandi with real safety intelligence. Free, open, and community-powered.
Create Free Account
Sign In
SafeRoute
© 2026 SafeRoute · Bhiwandi, India · Built for safety
Privacy Policy
Terms
Safety Tips
GitHub

2) Working Process

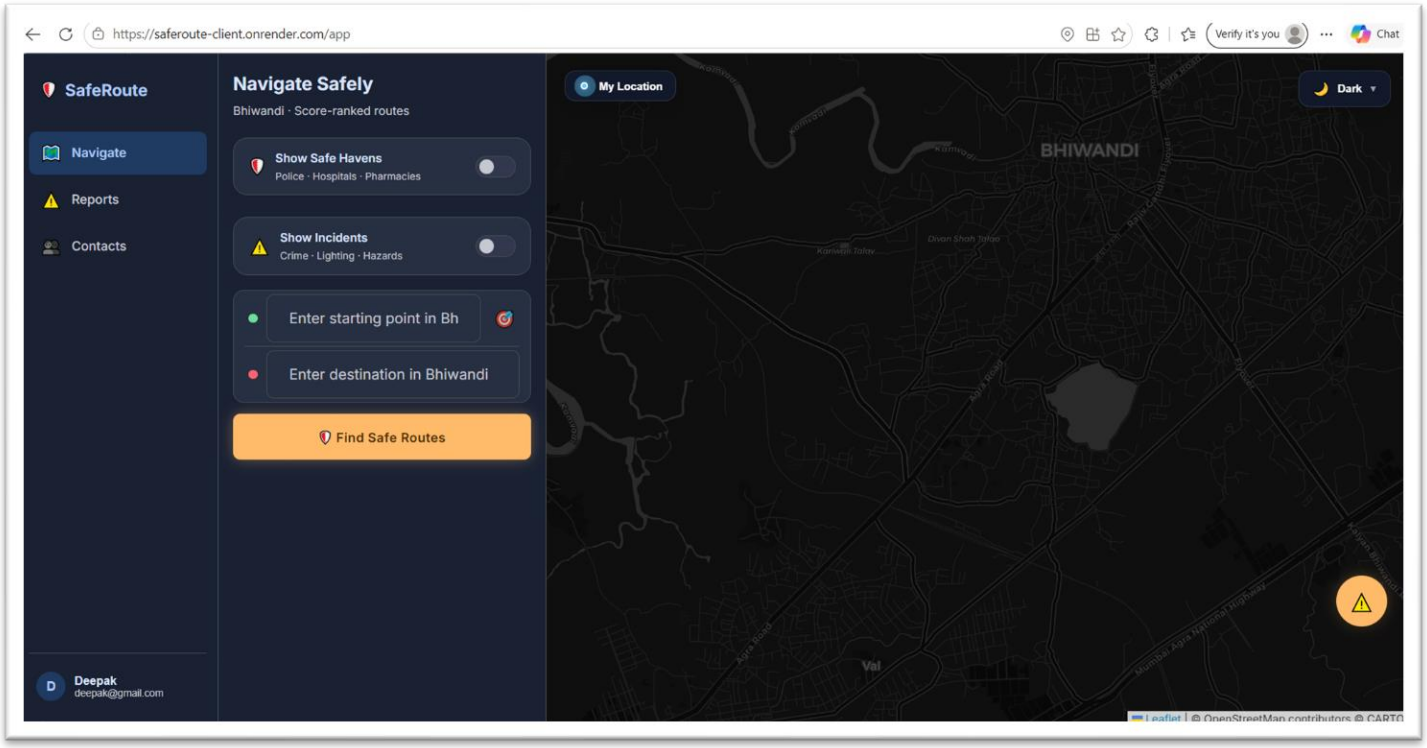


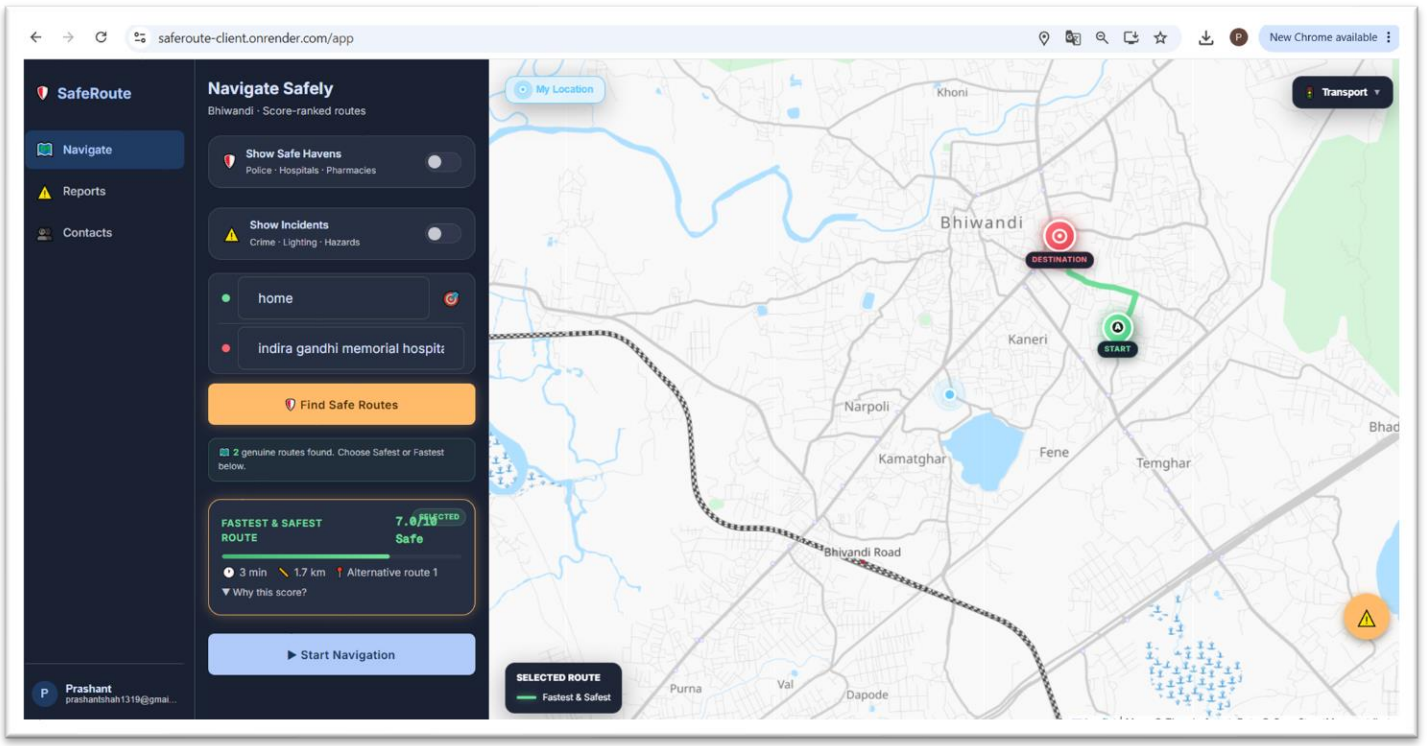


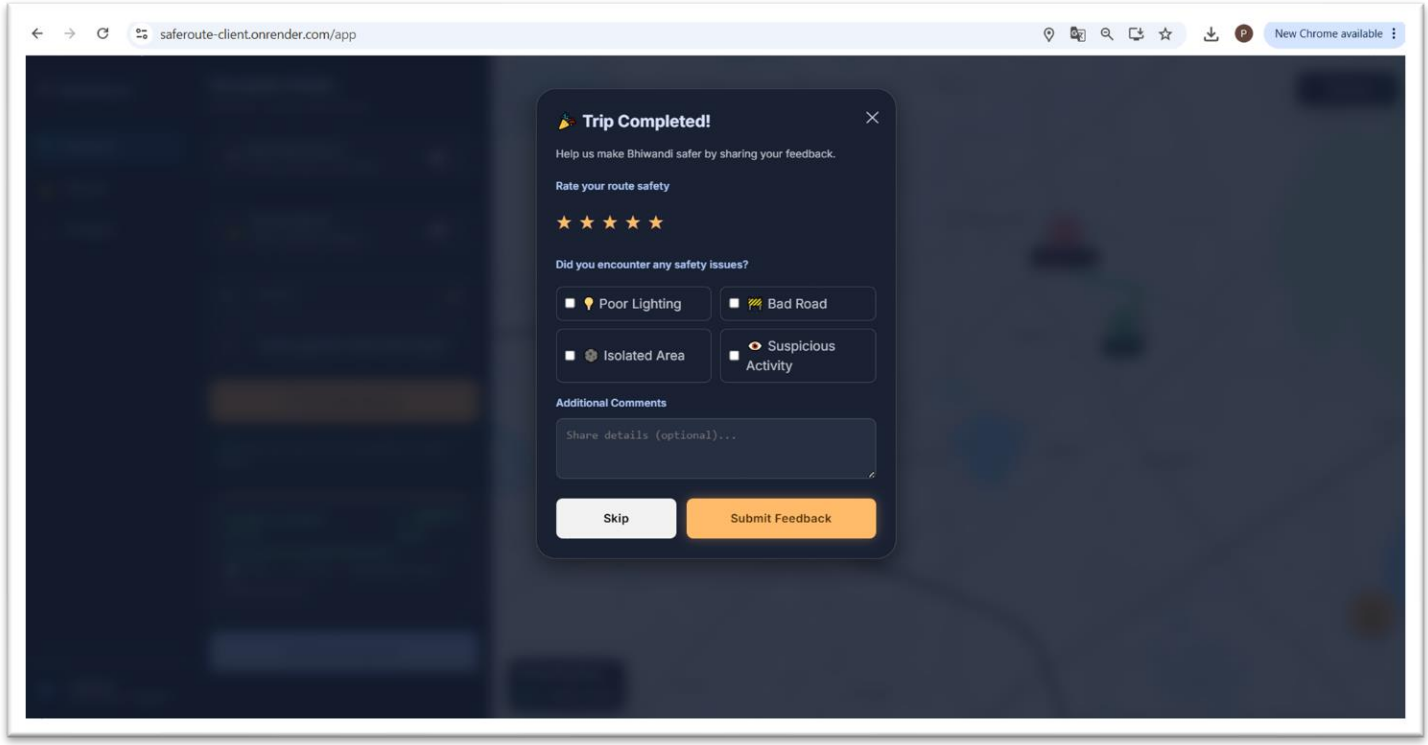


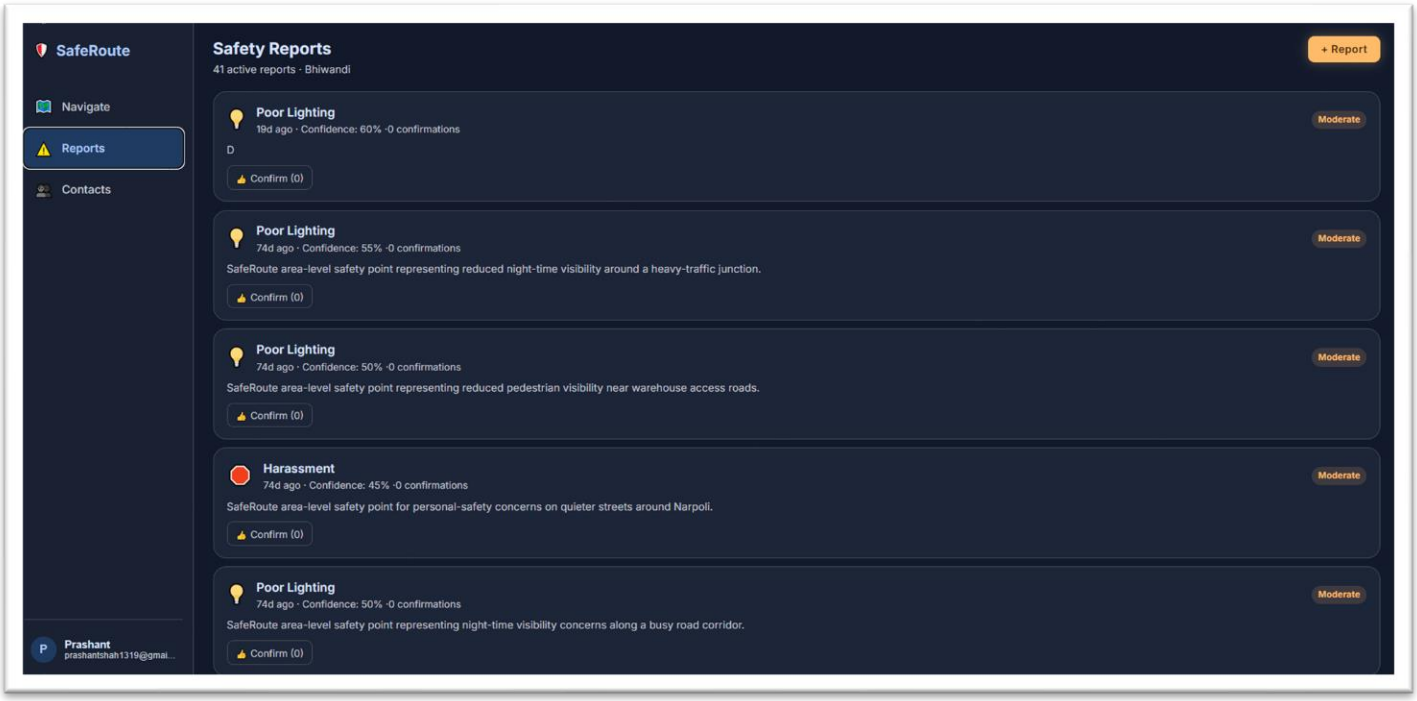


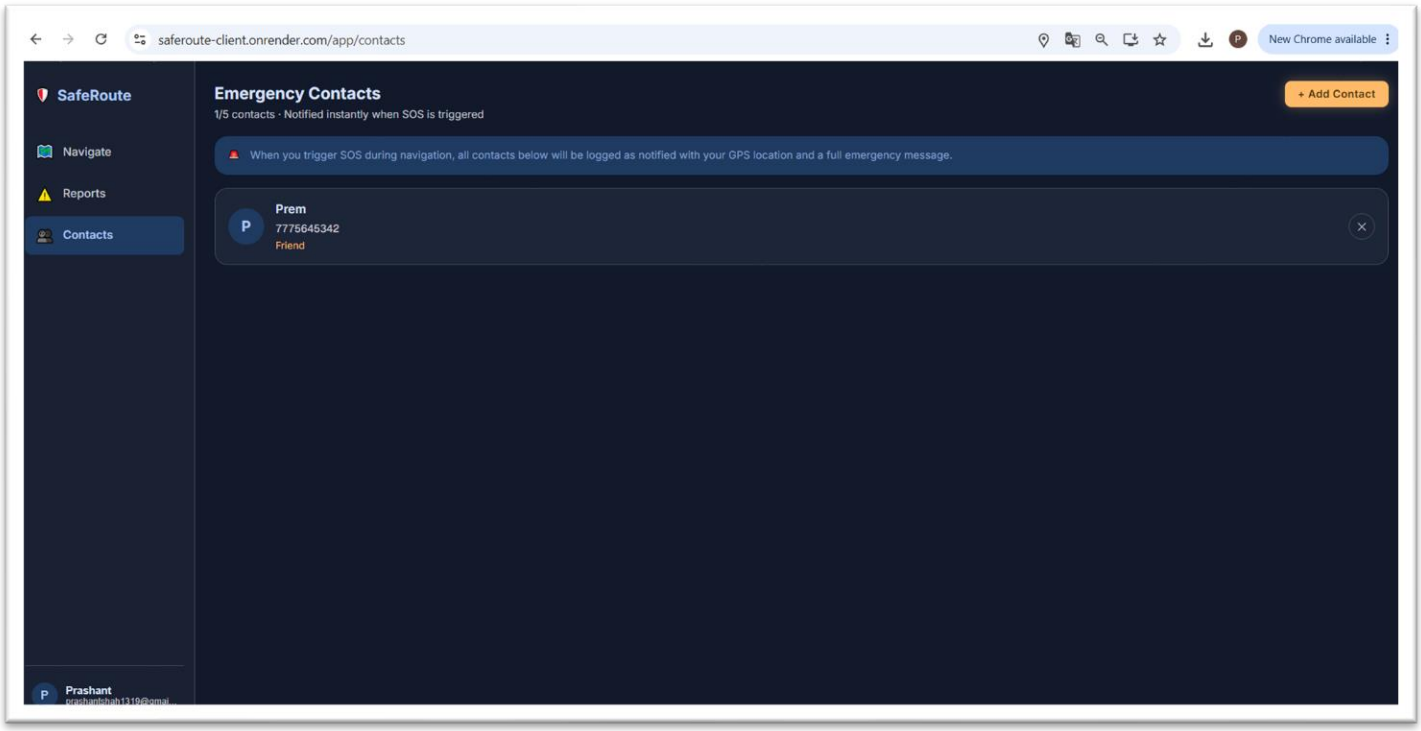
3) Main webpage Interface

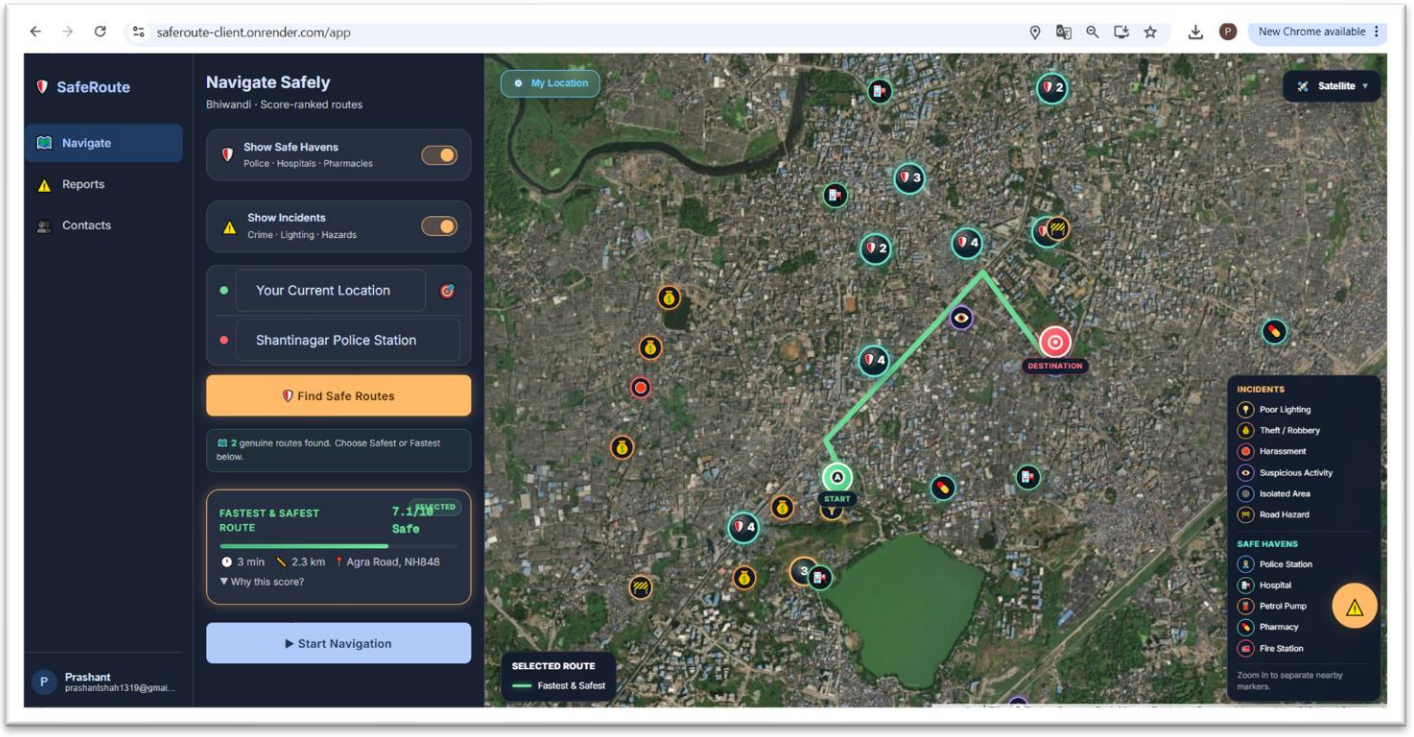





saferoute-client.onrender.com/app
New Chrome available
Trip Completed!
Help us make Bhiwandi safer by sharing your feedback.
Rate your route safety
Did you encounter any safety issues?
Poor Lighting
Bad Road
Isolated Area
Suspicious Activity
Additional Comments
Share details (optional)...
Skip
Submit Feedback


SafeRoute
Navigate
Reports
Contacts
Prashant
prashantshah1319@gmai...
Safety Reports
41 active reports · Bhiwandi
+ Report
Poor Lighting
19d ago · Confidence: 60% -0 confirmations
Moderate
D
Confirm (0)
Poor Lighting
74d ago · Confidence: 55% -0 confirmations
Moderate
SafeRoute area-level safety point representing reduced night-time visibility around a heavy-traffic junction.
Confirm (0)
Poor Lighting
74d ago · Confidence: 50% -0 confirmations
Moderate
SafeRoute area-level safety point representing reduced pedestrian visibility near warehouse access roads.
Confirm (0)
Harassment
74d ago · Confidence: 45% -0 confirmations
Moderate
SafeRoute area-level safety point for personal-safety concerns on quieter streets around Narpoli.
Confirm (0)
Poor Lighting
74d ago · Confidence: 50% -0 confirmations
Moderate
SafeRoute area-level safety point representing night-time visibility concerns along a busy road corridor.
Confirm (0)


saferoute-client.onrender.com/app/contacts
New Chrome available
SafeRoute
Navigate
Reports
Contacts
Emergency Contacts
1/5 contacts · Notified instantly when SOS is triggered
+ Add Contact
When you trigger SOS during navigation, all contacts below will be logged as notified with your GPS location and a full emergency message.
Prem
7775645342
Friend
Prashant


saferoute-client.onrender.com/app
New Chrome available
SafeRoute
Navigate
Reports
Contacts
Navigate Safely
Bhiwandi · Score-ranked routes
Show Safe Havens
Police · Hospitals · Pharmacies
Show Incidents
Crime · Lighting · Hazards
Your Current Location
Shantinagar Police Station
Find Safe Routes
2 genuine routes found. Choose Safest or Fastest below.
FASTEST & SAFEST ROUTE
Safe
3 min
2.3 km
Agra Road, NH848
Why this score?
Start Navigation
Prashant
My Location
Satellite
START
DESTINATION
INCIDENTS
Poor Lighting
Theft / Robbery
Harassment
Suspicious Activity
Isolated Area
Road Hazard
SAFE HAVENS
Police Station
Hospital
Petrol Pump
Pharmacy
Fire Station
Zoom in to separate nearby markers.
SELECTED ROUTE
Fastest & Safest

TESTING

Testing is performed to check whether SafeRoute works according to its requirements. It helps identify errors, verify the interaction between system components, and improve the reliability of the application.

5.1 Testing Approach

SafeRoute is tested using the following approaches.

1) **Unit Testing**
   Individual functions and components are checked separately. For example, input validation and route-scoring functions can be tested independently.
2) **Integration Testing**
   The interaction between the frontend, backend, database, and configured routing service is checked.
3) **Functional Testing**
   The main application features are tested to determine whether they behave as expected.
4) **User Interface Testing**
   Forms, buttons, maps, navigation controls, and messages are checked for usability and correct display.
5) **Error Handling Testing**
   Invalid input, incorrect login credentials, unavailable services, and failed requests are tested to check whether appropriate error messages are displayed.
6) **Route Analysis Testing**
   Available route options and safety-related data are checked to verify that the application produces results according to the implemented scoring method.

5.2 Functional Testing

Functional testing focuses on the main features of SafeRoute.

The following features should be checked:

- User registration and login
- Route search and display
- Comparison of available route options
- Safety-score calculation
- Identification of the safest available route
- Display of a faster, safety-conscious alternative when available
- Incident reporting
- SOS emergency interface
- Map display
- Nearby services
- Communication between the frontend and backend
- Database operations

Testing should confirm that the application responds correctly to valid and invalid user actions.

5.3 Test Cases and Results

The following table provides a sample testing format. Enter the actual result after executing each test in the application.

Test ID	Test case	Result
TC01	Register with valid details	Account is created successfully
TC02	Log in with valid credentials	User can access the application
TC03	Log in with invalid credentials	Appropriate error message is displayed
TC04	Search using valid origin and destination	Available route options are displayed
TC05	Search with invalid or unavailable locations	Appropriate validation or error message is displayed
TC06	Compare available routes	Route details and safety information are displayed
TC07	Calculate route safety scores	Scores are produced according to the implemented method
TC08	Identify the safest available route	The route with the best safety score under the implemented method is identified
TC09	Submit a valid incident report	The report is processed and saved successfully
TC10	Submit an invalid incident report	Validation or error message is displayed
TC11	Open the SOS feature	The implemented SOS interface or action is available
TC12	Open the map	The map displays correctly when the required service is available
TC13	View nearby services	Available service information is displayed
TC14	Test database connectivity	Required database operations succeed

5.4 Testing Summary

Testing helps determine whether the main features of SafeRoute function correctly and whether the components communicate properly.

The testing process covers user authentication, route search, route comparison, safety scoring, incident reporting, the SOS interface, maps, nearby services, and database connectivity.

Errors discovered during testing are corrected and the affected features tested again. The final testing summary includes the number of tests performed, the number passed, the number failed, and any remaining issues.

ADVANTAGES AND LIMITATIONS

6.1 Advantages

SafeRoute offers several advantages.

1) Safety-aware route planning: It considers available safety-related information when evaluating routes.

2) Multiple route options: It allows users to compare different available routes instead of relying on a single option.

3) Safest-route recommendation: It identifies the safest available route according to its implemented scoring method.

4) Travel-time consideration: It can provide a faster route that considers safety when a suitable option is available.

5) Incident reporting: Users can report safety-related problems in different locations.

6) Map-based display: Routes and geographical information are presented visually.

7) Nearby services: Users can access information about supported safe places and useful services.

8) User authentication: Login functionality supports access to protected features.

9) User-friendly interface: The application aims to make route selection simple and understandable.

10) Potential for improvement: Additional data sources and improved scoring methods can enhance the system in the future.

6.2 Limitations

Although SafeRoute provides useful features, it has certain limitations.

1) Data accuracy: Route recommendations depend on the accuracy and reliability of available incident information.

2) Outdated reports: Old incident reports may not reflect the current situation in an area.

3) Incomplete coverage: Some locations may have limited safety-related data.

4) No guarantee of safety: A high safety score does not guarantee that a route is completely safe.

5) External service dependency: Route and map information may be affected if a configured external service is unavailable.

6) Internet dependency: Online features may require an active internet connection.

7) Location accuracy: Incorrect or imprecise location information can affect route results.

8) User-reported information: Incident reports may contain errors or unverified information.

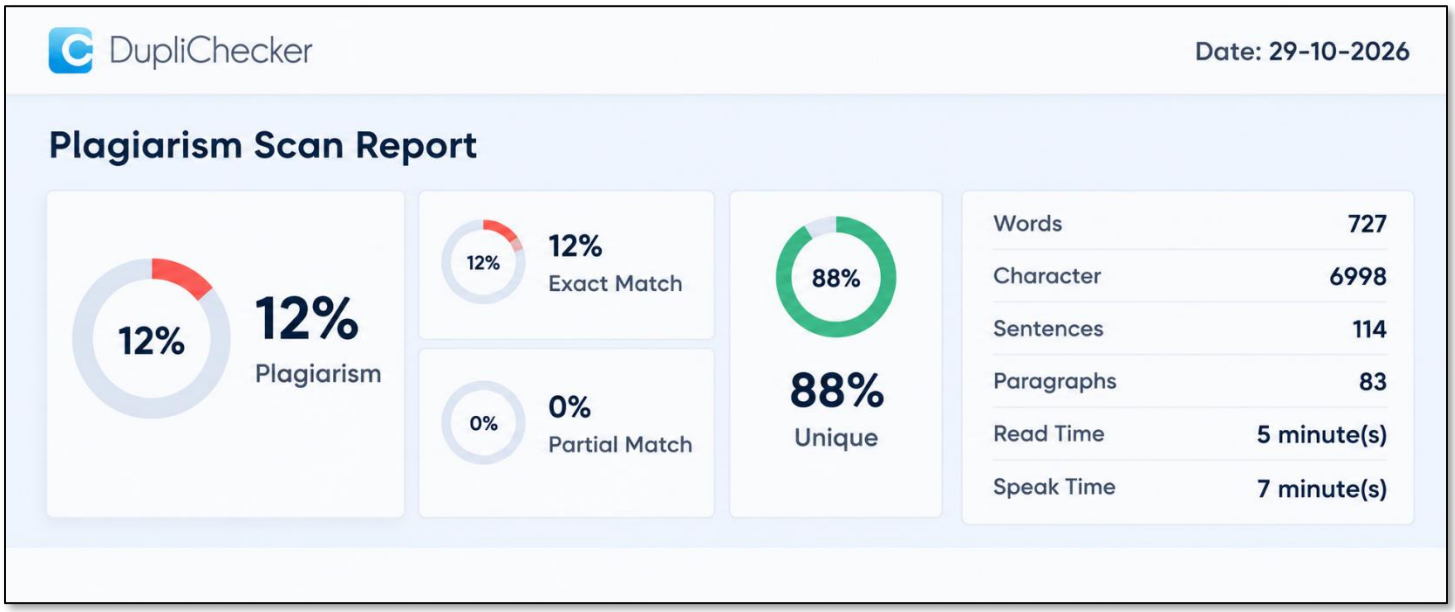
9) Limited emergency functionality: The SOS feature can provide only the actions that have actually been implemented.

10) Scoring limitations: The quality of route recommendations depends on the scoring method and the data available to the system.

These limitations can be reduced through better data quality, regular maintenance, improved validation, and further development.

PLAGIARISM REPORT

`



FUTURE SCOPE

As the developer of SafeRoute, I believe that the system has the potential to be improved further by adding new features and enhancing its existing functionality. The main aim of future development will be to make the application more reliable, user-friendly, and effective in helping users choose safer routes.

1) **Real-Time Safety Information**
   In the future, I would like to integrate trusted and regularly updated sources of safety-related information so that SafeRoute can analyse routes using more accurate and recent data.

2) **Improved Safety Scoring**
   I plan to improve the safety-scoring method by considering factors such as incident severity, distance from reported incidents, freshness of data, and the reliability of user-submitted reports. This could help provide more meaningful safety scores.

3) **Better Route Comparison**
   I would like to enhance the route comparison feature so that users can easily compare the fastest route, the safest available route, and alternative routes that offer a balance between safety and travel time.

4) **Dedicated Mobile Application**
   In future development, I plan to explore creating a dedicated Android and iOS application. This would provide a more convenient mobile experience and allow better integration with supported device features.

5) **Incident Report Verification**
   I would like to introduce a report-verification system to identify duplicate, incorrect, or misleading reports. This could improve the quality and reliability of the information used for route analysis.

6) **Enhanced SOS Emergency Feature**
   I plan to improve the SOS feature by exploring more reliable ways to notify saved emergency contacts and, where technically and legally supported, relevant emergency services. User consent and privacy will remain important considerations.

7) **Improved Live Location Sharing**
   In future versions, I would like to enhance Guardian Mode by making journey tracking and live location sharing more convenient for trusted contacts while ensuring that location sharing remains secure and controlled by the user.

8) **More Accurate Nearby-Service Information**
   I plan to improve the nearby-service feature by integrating reliable and regularly updated information about hospitals, police stations, pharmacies, and other useful locations.

9) **Personalised Route Preferences**
   I would like to provide users with more control over route selection by allowing them to prioritise safety, travel time, or distance according to their individual needs.

.

10) **Advanced Data Analysis**
    In the future, I plan to explore statistical analysis and machine-learning techniques to identify patterns in reliable historical incident data. These techniques could help improve route analysis, provided that suitable datasets are available and privacy, accuracy, and possible bias are carefully considered.

CONCLUSION

SafeRoute – Smart Safety-Aware Navigation System is a security-first, community-driven navigation application developed to help users make more informed travel decisions.

Unlike navigation systems that primarily focus on speed and distance, SafeRoute evaluates available route options using safety-related information. Its safety-scoring engine provides ratings on a scale of 1 to 10 according to the implemented method. The application helps users compare the fastest and safest available routes and identify a suitable option based on their preferences.

The project includes several important features, such as user registration and login, route search, safety-score calculation, community incident reporting, Guardian Mode, an SOS emergency workflow, nearby safe havens, and an installable Progressive Web App experience.

The application uses React, Vite, Tailwind CSS, Node.js, Express.js, MongoDB, Mongoose, Socket.IO, and mapping services such as OpenStreetMap, OSRM, Nominatim, and Overpass. These technologies work together to support the application's interface, data management, route retrieval, and real-time features.

The completed project has been tested and prepared for demonstration. The testing process covered the main application features and their interactions. SafeRoute demonstrates how web technologies, mapping services, and community-based information can be combined to support safety-aware navigation.

However, the reliability of route recommendations depends on the available data, and the safety score cannot guarantee that a route is completely safe. Real-time tracking and emergency features also depend on the correct operation of their configured communication mechanisms.

In conclusion, SafeRoute provides a practical foundation for navigation that considers safety alongside travel convenience. With further improvements to data quality, route analysis, and privacy controls, it has the potential to become an increasingly useful tool for students, commuters, tourists, and other safety-conscious travellers.

SafeRoute — Navigate Smart. Travel Safe.

REFERENCES

The following official documentation resources are used in SafeRoute development.

1) React Documentation. https://react.dev/
2) Vite Documentation. https://vite.dev/guide/
3) Tailwind CSS Documentation. https://tailwindcss.com/docs
4) Framer Motion Documentation. https://motion.dev/
5) Node.js Documentation. https://nodejs.org/docs/
6) Express.js Documentation. https://expressjs.com/
7) MongoDB Documentation. https://www.mongodb.com/docs/
8) Mongoose Documentation. https://mongoosejs.com/docs/
9) Leaflet Documentation. https://leafletjs.com/reference.html
10) React Leaflet Documentation. https://react-leaflet.js.org/
11) Socket.IO Documentation. https://socket.io/docs/v4/
12) JSON Web Tokens. https://jwt.io/introduction
13) OpenStreetMap. https://www.openstreetmap.org/
14) Open-Source Routing Machine (OSRM). https://project-osrm.org/
15) GitHub Documentation. https://docs.github.com/